BLOCKAPEX

PROSPEREUM PROTOCOL

FINAL AUDIT REPORT

Date: 17th Apr 2026

Prepared for: WealthHealth AI, Inc

About Blockpex

Founded in early 2021, BlockApex is a security-first blockchain consulting firm. We offer services in a wide range of areas including Audits for Smart Contracts, Blockchain Protocols, Tokenomics along with Invariant development (i.e test-suite) and Decentralized Application Penetration Testing. With a dedicated team of over 40+ experts dispersed globally, BlockApex has contributed to enhancing the security of essential software components utilized by many users worldwide, including vital systems and technologies.

BlockApex has a focus on blockchain security, maintaining an expertise hub to navigate this dynamic field. We actively contribute to security research and openly share our findings with the community. Our work is available for review at our public repository, showcasing audit reports and insights into our innovative practices.

To stay informed about BlockApex's latest developments, breakthroughs, and services, we invite you to follow us on Twitter and explore our GitHub. For direct inquiries, partnership opportunities, or to learn more about how BlockApex can assist your organization in achieving its security objectives, please visit our "Contact" page at our website, or reach out to us via email at hello@blockapex.io.

Table of Contents

About Blockpex 1
Table of Contents 2
Project Overview 4
Scope & Version 5
Severity Definitions 6
Status Definitions 7
Methodology 8
Key Focus Areas 9
Summary of Findings 10
Status Overview: 10
Detailed Findings 13
Critical Severity Issues 13
- Issue #1: Staker Reward Over-Distribution via Post-Snapshot recordStakeTime 13
- Issue #2: Nested O(V×C) Loop Complexity in finalizeEpoch causes OOG DOS 16
- Issue #3: PartnerVault allows Flash Loan Inflation of cumS High-Water Mark via Direct Transfers 19
- Post Fixes Issues 22
- Issue #4: The ClaimStake Paradox (100% Retail Liveness Failure) 22
- Issue #5: Atomic Batching Annihilates Staking Yield 23

High Severity Issues 24
- Issue #6: createVault Economic Gate Bypass via Flash-Loaned Self-Sandwiching 24
- Issue #7: Cross-Epoch StakeTime Accumulation Leaks Into Wrong Epoch Total 27
- Issue #8: createVault S_MIN Economic Gate Bypass via Unvalidated Fee Tier Routing 30
- Post Fixes Issues 33
- Issue #9: autoFinalizeEpochs scarcity × K can exceed PSRE MAX_MINT_PER_EPOCH → DoS on vault operations 33
- Issue #10: StakingVault settlement breaks invariant , retroactive reward theft, reward loss, and over-payment 36

Medium Severity Issues 43
- Issue #11: PartnerVault.snapshotEpoch Compresses Multi-Epoch Growth Into Single Epoch, Causing Permanent Reward Loss via Cap 43
- Issue #12: Static partnerOf Mapping Leads to Reward Lockout and Unauthorized CV Deployment After Ownership Transfer 46
- Issue #13: recordStakeTime Cap Based on Current Balance Zeroes Rewards After Unstaking 49
- Issue #14: CustomerVault.reclaimUnclaimed is Permanently Unreachable Due to Missing Caller in PartnerVault 52
- Issue #15: customerVaultList Grows Monotonically Without Deregistration, Permanent Registration Cap DoS 54
- Issue #16: allVaults Array Is Append-Only With No Removal Path — Permanent Slot Exhaustion and Gas Ratchet 57
- Issue #17: LP Tokens Weighted 1:1 With PSRE in StakeTime Computation Enables Cheap Reward Dilution 60

Issue #18: transferOut Guard Bypassed via CustomerVault Destinations Enabling Cross-Vault cumS Inflation....62
Issue #19: Non-Recording Staker Checkpoints Inflate totalStakeTime Denominator, Diluting Rewards and Locking Minted PSRE.... 65
Issue #20: Independent PSRE and RewardEngine Pause States Create “Ghost” Emission Consumption....68
Issue #21: Compressed Multi-Epoch Δ effectiveCumS Spikes EMA Score, Granting Undeserved Tier Promotion....71
Post Fixes Issues....72
Issue #22: autoFinalizeEpochs() Scaled Minting Collides with PSRE Epoch Mint Cap, Permanent Protocol DoS....72
Issue #23: Stranded PSRE when an epoch is finalized with zero stakers on both sides....77
Low Severity Issues.... 80
Issue #24: RewardEngine.pause() Does Not Gate claimPartnerReward and claimStake, Emergency Freeze Cannot Prevent Reward Outflows.... 80
Issue #25: In-Loop sumR Mutation Causes Order-Dependent Tier Assignments....82
Issue #26: Multi-Epoch Accumulated StakeTime Permanently Destroyed by recordStakeTime's Blanket accStakeTime Reset.... 86
Issue #27: PartnerVault.qualified Flag Permanently False Due to Lack of Synchronization with RewardEngine....88
Post Fixes Issues....91
Issue #28: Stale partnerOwner in CustomerVault after vault ownership transfer....91
Informational Severity Issues....92
Issue #29: Governance Parameter Queue Functions Allow Silent Value Overwrite Without Cancellation Event.... 92
Disclaimer.... 94

Project Overview

Prospereum is a decentralized behavioral mining protocol that introduces Proof of Prosperity, where token issuance is driven by verifiable, sustained growth in ecosystem demand rather than passive holding or raw transaction volume. Instead of rewarding activity or time-based staking, the protocol measures cumulative net growth in PSRE holdings across a partner's vault ecosystem, ensuring that rewards are tied strictly to genuine economic contribution. This design eliminates common exploit vectors such as wash trading or cyclical capital reuse, as any reduction in holdings raises the threshold for future rewards rather than resetting it.

The protocol is built around a vault-based architecture, where partners manage on-chain PartnerVaults and deploy CustomerVaults for end users, enabling seamless, blockchain-agnostic participation. A central RewardEngine contract governs epoch-based emissions using a high-water-mark model (cumS) combined with a decompounding mechanism (effectiveCumS) to prevent reward inflation or recursive yield. Emissions are further constrained by a scarcity function, ensuring long-term supply discipline while aligning incentives across partners, customers, and stakers.

Scope & Version

Item	Miscellaneous
Project Name	Propereum Protocol
Scope	Repository • https://github.com/wealthhealthai/prospereum-protocol/tree/master Initial Commit Hash • 7e96ba968d2f3ca8259c39b47610885d4e65aa75 Revised Commit Hash • 2073cfe7ed0bc119f5240bde39fd909dd8a48122 Folder • https://github.com/wealthhealthai/prospereum-protocol/tree/master/contracts Branch • master Fixed Commit Hash • 31eb31384dee7385b14b1f02ac033e2e488e721f
Review Type	Comprehensive Manual Code Review
Initiation Date	2nd April 2026
Delivery Date	5th April 2026
Version	v1.1.0
Phase	Phase #2
Re audit Date	8th April 2026 - 17th April 2026

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on BlockApex's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
CRITICAL	vulnerabilities are too easy to exploit and can lead to damages/loss in assets or manipulations.
HIGH	vulnerabilities are difficult to exploit; however, they also have a significant impact on smart contract execution.
MEDIUM	vulnerabilities are equally imperative to fix but they tend to have minimal impact on asset loss or data manipulations.
LOW	vulnerabilities, informational errors, violating code styles/practices usually can't affect smart contract execution; hence they can be ignored.
INFORMATIONAL	Concerns that have no direct negative impact. These may help improve coding practices.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The reported issue has been addressed and verified as resolved.
Partially Resolved	Some remediation steps have been taken and risk may be reduced, but the vulnerability isn't fully resolved.
Acknowledged	The client has reviewed the finding and agrees it's valid, but has not implemented a fix (or plans to handle it later).
Open	The issue is still present and no remediation has been applied yet.

Methodology

The codebase was audited using a filtered audit technique. A band of two (2) auditors scanned the codebase in an iterative process for a time spanning 1 week. Starting with the recon phase, a basic understanding was developed, and the auditors worked on developing presumptions for the developed codebase and the relevant documentation/whitepaper. Furthermore, the audit moved on with the manual code reviews to find logical flaws in the codebase complemented with code optimizations, software, and security design patterns, code styles and best practices

Discovery & Familiarization

The process begins with a thorough analysis of the project's white paper, technical specifications, and full codebase to understand the underlying protocol, architecture, and dependencies. This initial step helps map out potential risk areas before detailed testing begins.

Automated & Static Analysis

Our security experts leverage industry-leading tools such as Slither, Surya, Oyente, and Mythril for static analysis. These tools, combined with advanced analyzers like model checkers, theorem provers, and custom scripts, provide an extensive first pass at detecting vulnerabilities.

Manual Review & Threat Modeling

The cornerstone of our methodology is an exhaustive manual review. We perform line-by-line code inspections, assess module interactions, and conduct threat modeling to identify logic flaws, design weaknesses, and attack surfaces that automated tools may miss.

Dynamic & Fuzz Testing

To expose hidden edge cases, BlockApex employs fuzzing frameworks such as Ethpilot, Vultron, ReGuard, Foundry, and Echidna. This helps uncover vulnerabilities related to unpredictable inputs, unexpected states, and real-world usage patterns.

Key Focus Areas

Design & Architecture

Overall protocol logic, upgrade paths, trust assumptions, and economic/game-theory soundness.

Access & State Control

Roles, permissions, key management, and protection of contract state and storage integrity.

Logic & Execution Flow

Arithmetic correctness, transaction ordering, reentrancy, and other runtime behaviors.

External Interaction

Safe handling of cross-contract calls, token standards, oracles, bridges, and third-party dependencies.

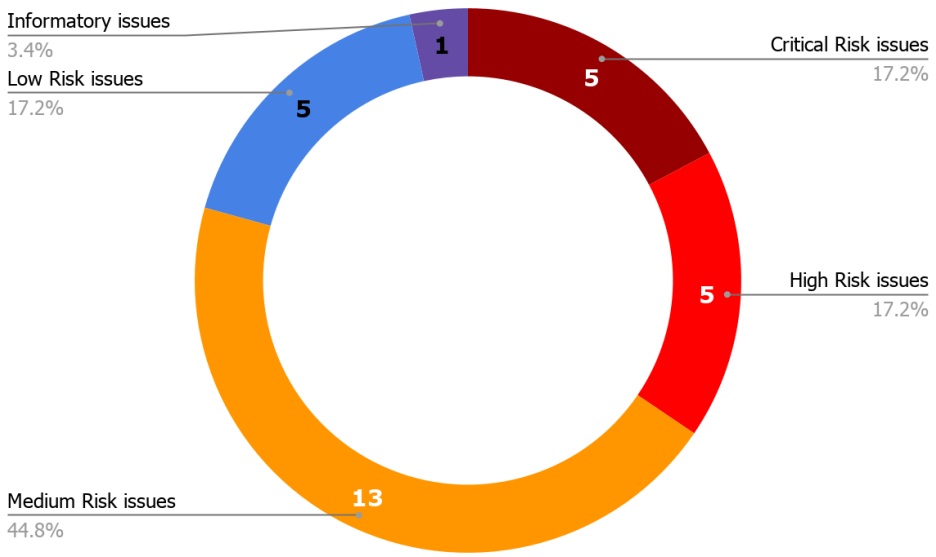
Lifecycle & Operations

Deployment security, upgrade/migration processes, monitoring, testing, and incident response readiness.

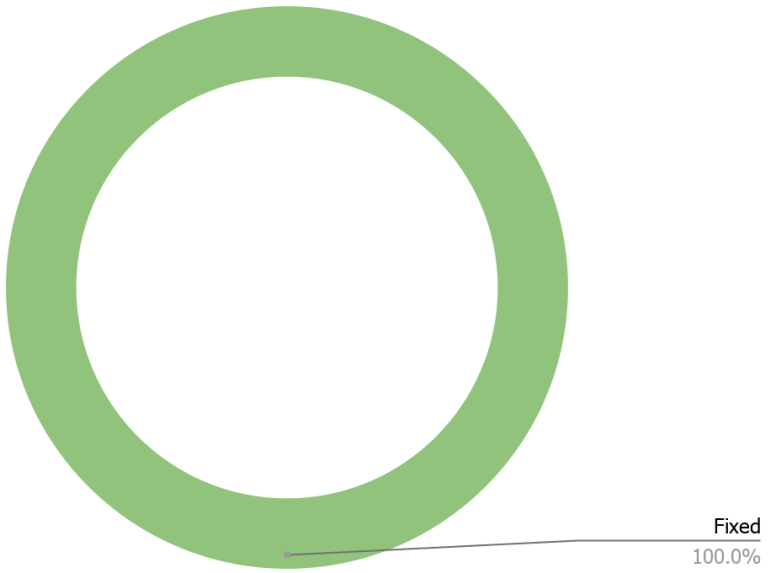
Summary of Findings

Our team conducted a Filtered Audit, in which two individuals independently reviewed the Propereum protocol contracts. The process involved thorough and rigorous manual testing, coupled with static analysis and including a line-by-line code review to identify potential bugs. All findings were carefully verified and re-tested to ensure their validity.

Issues Overview:



Status Overview:



Issue #	Title	Severity	Status
1	Staker Reward Over-Distribution via Post-Snapshot recordStakeTime	CRITICAL	Resolved
2	Nested O(V×C) Loop Complexity in finalizeEpoch causes OOG DOS	CRITICAL	Resolved
3	PartnerVault allows Flash Loan Inflation of cumS High-Water Mark via Direct Transfers	CRITICAL	Resolved
4	The ClaimStake Paradox (100% Retail Liveness Failure)	CRITICAL	Resolved
5	Atomic Batching Annihilates Staking Yield	CRITICAL	Resolved
6	createVault Economic Gate Bypass via Flash-Loaned Self-Sandwiching	HIGH	Resolved
7	Cross-Epoch StakeTime Accumulation Leaks Into Wrong Epoch Total	HIGH	Resolved
8	createVault S_MIN Economic Gate Bypass via Unvalidated Fee Tier Routing	HIGH	Resolved
9	autoFinalizeEpochs scarcity × K can exceed PSRE MAX_MINT_PER_EPOCH → DoS on vault operations	HIGH	Resolved
10	StakingVault settlement breaks invariant retroactive reward theft, reward loss, and over-payment	HIGH	Resolved
11	PartnerVault.snapshotEpoch Compresses Multi-Epoch Growth Into Single Epoch, Causing Permanent Reward Loss via Cap	MEDIUM	Resolved
12	Static partnerOf Mapping Leads to Reward Lockout and Unauthorized CV Deployment After Ownership Transfer	MEDIUM	Resolved
13	recordStakeTime Cap Based on Current Balance Zeroes Rewards After Unstaking	MEDIUM	Resolved
14	CustomerVault.reclaimUnclaimed is Permanently Unreachable Due to Missing Caller in PartnerVault	MEDIUM	Resolved
15	customerVaultList Grows Monotonically Without Deregistration, Permanent Registration Cap DoS	MEDIUM	Resolved
16	allVaults Array Is Append-Only With No Removal Path — Permanent Slot Exhaustion and Gas Ratchet	MEDIUM	Resolved

17	LP Tokens Weighted 1:1 With PSRE in StakeTime Computation Enables Cheap Reward Dilution	MEDIUM	Resolved
18	transferOut Guard Bypassed via CustomerVault Destinations Enabling Cross-Vault cumS Inflation	MEDIUM	Resolved
19	Non-Recording Staker Checkpoints Inflate totalStakeTime Denominator, Diluting Rewards and Locking Minted PSRE	MEDIUM	Resolved
20	Independent PSRE and RewardEngine Pause States Create “Ghost” Emission Consumption	MEDIUM	Resolved
21	Compressed Multi-Epoch Δ effectiveCumS Spikes EMA Score, Granting Undeserved Tier Promotion	MEDIUM	Resolved
22	autoFinalizeEpochs() Scaled Minting Collides with PSRE Epoch Mint Cap, Permanent Protocol DoS	MEDIUM	Resolved
23	Stranded PSRE when an epoch is finalized with zero stakers on both sides	MEDIUM	Resolved
24	RewardEngine.pause() Does Not Gate claimPartnerReward and claimStake, Emergency Freeze Cannot Prevent Reward Outflows	LOW	Resolved
25	In-Loop sumR Mutation Causes Order-Dependent Tier Assignments	LOW	Resolved
26	Multi-Epoch Accumulated StakeTime Permanently Destroyed by recordStakeTime's Blanket accStakeTime Reset	LOW	Resolved
27	PartnerVault.qualified Flag Permanently False Due to Lack of Synchronization with RewardEngine	LOW	Resolved
28	Stale partnerOwner in CustomerVault after vault ownership transfer	LOW	Resolved
29	Governance Parameter Queue Functions Allow Silent Value Overwrite Without Cancellation Event	INFO	Resolved

Detailed Findings

Critical Severity Issues

Issue #1: Staker Reward Over-Distribution via Post-Snapshot `recordStakeTime`

Severity: CRITICAL

Description:

A critical mathematical decoupling occurs in the `StakingVault` contract between the global `totalStakeTime` tracking mechanism and an individual user's `stakeTime` allocation. Because of a missing state-check in `recordStakeTime()`, an attacker can legally bypass the global tracking pool, recording a mathematically colossal individual multiplier that drains the engine of tokens it does not owe.

During standard operations, `RewardEngine.finalizeEpoch()` triggers a global snapshot locking in the total staked time accrued by honest users across the week:

From `StakingVault.sol:`

```
None
function snapshotEpoch(uint256 epochId) external {
    // ...
    epochSnapshotted[epochId]          = true;
    totalStakeTimeByEpoch[epochId]     = currentEpochTotalStakeTime;
    currentEpochTotalStakeTime         = 0; // The aggregator resets for the next
week
}
```

However, the user-facing function `recordStakeTime()` was programmed without enforcing that it must be triggered before this snapshot occurs.

```
None

function recordStakeTime(uint256 epochId) external {
    // Only verifies the timer has ended:
```

```
    require(currentEpochId() > epochId, "StakingVault: epoch not ended");

    // Missing check: require(!epochSnapshotted[epochId]);

    _checkpointUser(msg.sender); // Automatically adds 7 days of time for the
attacker!

    // ... calculates and successfully saves the attacker's massive 7-day delta.
    userStakeTimeByEpoch[epochId][msg.sender] = st;
}
```

By calling `recordStakeTime` after the snapshot, the attacker forces their massive 7-day stake tracking to apply retroactively to their individual record for the finalized epoch, while totally avoiding inclusion in the finalized global `totalStakeTimeByEpoch` denominator.

Proof of Concept:

The vulnerability can be successfully exploited in 5 atomic steps:

1. **Silent Staking:** At the beginning of Epoch , the attacker stakes a large amount of PSRE. They do not interact with the contract at all for the next 7 days, meaning `_checkpointUser` is never triggered to add their time to `currentEpochTotalStakeTime`.
2. **Global Snapshot:** Exactly 7 days later, `finalizeEpoch()` is called. The snapshot locks in a very tiny denominator (e.g., 100 PSRE staked by honest, active users). The denominator `totalStakeTime` is explicitly frozen permanently at `100 * 7 days`.
3. **Retroactive Injection:** The attacker instantly calls `recordStakeTime(EPOCH_ID)`. Because it lacks snapshot protections, the function computes their 7-day elapsed time, yielding a massive `10,000 * 7` days limit, and saves it into their isolated user record `userStakeTimeByEpoch[(EPOCH_ID]`.
4. **The Drain Equation:** The attacker calls `claimStake((EPOCH_ID)`. The `RewardEngine` utilizes a proportional fraction to mint rewards:

```
None
// From RewardEngine.sol:
uint256 reward = (epochStakersPool[epochId] * userST) / totalST;
```

Plugging in the attacker's values: `reward = Pool * (10,000 * 7 days) / (100 * 7 days) reward = Pool * 100,`

5. **Impact Realized**: The math calculates that the attacker owns 100x the size of the Epoch pool. The `safeTransfer` triggers directly, sweeping all available un-claimed PSRE historically held in the `RewardEngine` vault (which serves as an escrow for previous un-claimed Partner and Staker rewards).

Impact:

An attacker with sufficient capital can intentionally decouple their tracking pool to achieve a multiplier that cleanly drains the principal Token balance out of the Central `RewardEngine` contract.

Recommendation:

Immediate Fix: Add a strict state restriction to prevent post-snapshot execution.

```
None
function recordStakeTime(uint256 epochId) external {
    require(currentEpochId() > epochId, "StakingVault: epoch not ended");
+   require(!epochSnapshotted[epochId], "StakingVault: epoch already
snapshotted");
    require(userStakeTimeByEpoch[epochId][msg.sender] == 0, "StakingVault: already
recorded");
}
```

CRITICAL Architectural Warning: Patching this logic flaw introduces an impossibly hostile UX/UI race condition for organic humans. By enforcing `require(!epochSnapshotted)`, users must manually call `recordStakeTime` in the tiny microscopic delay between when the epoch timer hits 0, and when a Keeper bot calls `finalizeEpoch`. If the user is asleep and misses this fraction-of-a-second window, they will permanently forfeit their weekly rewards.

Long-Term Resolution: The Staking architecture should be thoroughly refactored to remove the manual `recordStakeTime` requirement. Standard Staking modules (like the Synthetix implementation) perform Time-Weighted distributions implicitly inside stake/unstake/claim paths, rather than forcing users to manually flush accumulators into epochs.

Status: Resolved

commit 6a3dda8

Issue #2: Nested O(V×C) Loop Complexity in finalizeEpoch causes OOG DOS

Severity: CRITICAL

Description:

The `RewardEngine.finalizeEpoch()` function is the central mechanism for advancing the protocol's state and distributing PSRE. It contains a severe algorithmic vulnerability via an unbounded nested loop structure that inherently breaks the EVM block gas limit.

When `finalizeEpoch` is called, an outer loop iterates over every registered `PartnerVault` (capped at 200 by the Factory).

```
None
// From RewardEngine.finalizeEpoch()
uint256 nVaults = allVaults.length; // Up to 200
for (uint256 i = 0; i < nVaults; i++) {
    address vault = allVaults[i];
    uint256 deltaEffectiveCumS = IPartnerVault(vault).snapshotEpoch();
}
```

Inside this loop, the `IPartnerVault(vault).snapshotEpoch()` function unconditionally triggers an inner loop in `PartnerVault._updateCumS()` which iterates over every attached `CustomerVault` (capped at 1,000 per partner).

To calculate the physical total balance, the inner loop executes an external cross-contract token `balanceOf()` call for each CustomerVault.

```
None
// From PartnerVault._updateCumS()
uint256 nCVs = customerVaultList.length; // Up to 1,000
uint256 currentTVL = 0;
for (uint256 i = 0; i < nCVs; i++) {
    // External condition: ~2,600 EVM Gas per execution!
    currentTVL += IERC20(psre).balanceOf(customerVaultList[i]);
}
```

Because the outer loop runs 200 times and the inner loop runs 1,000 times, this creates a worst-case EVM computational load of `200 * 1000 = 200,000` external read operations. To complete, the transaction requires over ~520 million EVM Gas, which completely shatters the Base network protocol limitations. Base have per tx limit of 25 Million.

Proof of Concept:

1. **The Set Up:** A malicious competitor (or simply organic protocol adoption) registers 10 to 12 Partner Vaults.
2. **Customer Vault Inflation:** For each Partner Vault, they deploy and attach the maximum 1,000 Customer Vaults allowed by the factory constraints.
3. **The DoS Trigger:** The 7-day epoch timer completes, and the Keeper Bot attempts to call `finalizeEpoch()`.
4. **Execution Failure:** The transaction enters the nested loop mechanism. With just 12 maxed-out Partner Vaults, the EVM must perform 12,000 external `balanceOf` reads.
5. **Impact Realized:** The gas consumed rapidly scales to over `31,000,000` gas. Base L2 enforces strict block and per-transaction limits on EVM computation (often between ~15M - 30M per tx). The sequencer rejects the transaction, reverting with a fatal `Out of Gas (OOG)` error.

Impact:

`finalizeEpoch` has no mechanism to "skip" vaults or save partial progress, the function must process ALL vaults atomically in a single transaction. Once the ecosystem naturally scales past roughly 12-20 active maxed partners (or if an attacker spends ~$2000 to deploy dummy vaults), the function permanently bricks. The protocol enters an inescapable Denial of Service (DoS) state, permanently freezing the entire reward economy.

Note:
With the introduction of autoFinalizeEpochs() which is triggered upon buy() and createVault() this turn into an DOS vector where 10 pending epoch coupled with O(VxC) triggers a DOS on the protocol.

Recommendation:

The protocol cannot afford to utilize ***O(N x M)*** atomic algorithms on mainnet.

Immediate Refactor: Implement pagination (chunking) in `RewardEngine.finalizeEpoch()`. Instead of looping `nVaults` universally in a single transaction, breaking the process up allows the Keeper to finalize chunks of vaults across multiple transactions/blocks before officially rolling over the epoch. Moreover the architecture should be refactored.

Status: Resolved

commit 7c13690

Issue #3: PartnerVault allows Flash Loan Inflation of `cumS` High-Water Mark via Direct Transfers

Severity: CRITICAL

Description:

The `PartnerVault` contract uses a "high-water mark" mechanism (`cumS`) to track the peak ecosystem balance for reward calculation. The `_updateCumS()` function implements this by scanning the live ERC-20 balances of the `PartnerVault` and its registered `CustomerVaults`. If the live balance exceeds the current `ecosystemBalance`, the `cumS` ratchet is updated upward.

This design constitutes a critical vulnerability because it allows an attacker to use a flash loan to artificially spike the `cumS` value. Since `cumS` is a monotonically increasing ratchet (meaning it never decreases, even when funds are withdrawn), an attacker can inject a massive amount of PSRE into the vault within a single transaction, trigger an update to the high-water mark, and then return the flash-loaned funds.

The `RewardEngine` will then treat this artificial peak as genuine growth and mint a disproportionately large amount of rewards for the attacker.

Affected Code Snippets:

1. **The vulnerable balance scan in `PartnerVault.sol`**

None

```
// contracts/core/PartnerVault.sol:379-398
function _updateCumS() internal {
    uint256 ownBalance = IERC20(psre).balanceOf(address(this));
    uint256 totalEcosystem = ownBalance;
    uint256 n = customerVaultList.length;

    for (uint256 i = 0; i < n; i++) {
        totalEcosystem += IERC20(psre).balanceOf(customerVaultList[i]);
    }

    if (totalEcosystem > ecosystemBalance) {
        ecosystemBalance = totalEcosystem;
    }
```

```
    if (ecosystemBalance > cumS) {
        cumS = ecosystemBalance; // The ratchet is spiked here
    }
}
```

2. **Triggering the update via `buy()`**

```
None

// contracts/core/PartnerVault.sol:224
function buy(...) external onlyOwner ... {
    // ... swap logic ...
    ecosystemBalance += psreOut;
    _updateCumS(); // Triggers the flash-loan capture
}
```

Proof of Concept:

1. **Flash Loan:** An attacker takes a flash loan of 10,000,000 PSRE.
2. **Injection:** The attacker transfers the 10,000,000 PSRE directly to their `PartnerVault`.
3. **Trigger:** The attacker calls `buy()` with a minimal amount of USDC.
   A. `_updateCumS()` is executed.
   B. `totalEcosystem` is detected as 10,000,000+.
   C. `cumS` is ratcheted up to 10,000,000+.
4. **Recovery:** The attacker immediately calls `transferOut(10,000,000)` to return the flash-loaned tokens.
5. **Ratchet Persistence:** The `ecosystemBalance` returns to normal, but the cumS remains at 10,000,000+.
6. **Reward Extraction:** At the end of the epoch, `RewardEngine.finalizeEpoch` calls `snapshotEpoch()`, which returns a massive `deltaCumS`. The attacker is awarded a huge amount of newly minted PSRE.

Impact:

This vulnerability allows an attacker to extract a massive portion of the total protocol emission budget without any long-term capital commitment. By manipulating the `cumS` ratchet, a single attacker can effectively "drain" the reward pool for all other legitimate partners, potentially bankrupting the reward system in a single epoch.

Recommendation:

Remove the dependency on live `balanceOf` reads for updating the `cumS` high-water mark. Instead, the system should only track growth that occurs through explicit, tracked deposits.

- **Disable Live Balance Scanning:** Remove the `IERC20(psre).balanceOf` calls from `_updateCumS`.
- **Implement Explicit Deposits:** Create a `depositPSRE()` function that explicitly updates `ecosystemBalance` and `cumS`. This allows the protocol to implement safeguards (like a minimum lockup period) to ensure that the funds used to move the high-water mark are not flash-loaned.
- **Strict Accounting:** Only allow `cumS` to grow based on transactions that are recorded and verified as "committed" capital.

Status: Resolved

commit 79bbaa8

Post Fixes Issues

Issue #4: The ClaimStake Paradox (100% Retail Liveness Failure)

Severity: CRITICAL

Description:

The atomic batching issue highlighted in Vulnerability 2 is drastically compounded by a fatal logic paradox in the reward claiming mechanism. The current architecture guarantees that 100% of retail users who attempt to claim their rewards natively will lose them.

The Execution Flow of a Normal User:

1. A user stakes tokens for Epoch 1. They wait until the Epoch finishes and the protocol is finalized.
2. RewardEngine calls snapshotEpoch(1), finalizing the epoch and locking it: epochSnapshotted[1] = true.
3. The user calls claimStake(1).

Inside claimStake, two conflicting conditions occur at the exact same time:

```
None

function claimStake(uint256 epochId) external nonReentrant {
    // CONDITION 1: Requires that the Epoch IS ALREADY snapshotted
    require(epochSnapshotted[epochId], "StakingVault: epoch not finalized");

    // Attempt to credit the user's unrecorded elapsed time
    _checkpoint(msg.sender);
```

Inside _checkpoint, the code calculates all the time the user spent in the vault and routes it to _addContribution():

```
None
function _addContribution(...) internal {
    // CONDITION 2: Requires that the Epoch IS NOT snapshotted
    if (epochSnapshotted[epochId]) return;
```

Because claimStake() explicitly verified that epochSnapshotted[epochId] was true, the _addContribution function instantly returns and silently throws all of the user's elapsed time into the trash. Their userPSREStakedTime[epochId] remains 0.

The claimStake function resumes, sees that the user has 0 accumulated time, and reverts the entire transaction with "StakingVault: nothing to claim".

The only viable way for users to earn yields is if an automated Keeper bot pays O(N) network gas to blindly iterate through and manually call checkpointUser() on every single user's address exactly before the snapshot occurs, which is economically unviable and structurally flawed.

Status: Resolved
commit 177c18e

Issue #5: Atomic Batching Annihilates Staking Yield

Severity: CRITICAL

Description:

To compensate for the Partner rewards, autoFinalizeEpochs() processes all missed Epochs simultaneously in a single atomic while loop. This loop sequentially calls _finalizeSingleEpoch(1), (2), and (3).

This causes StakingVault.snapshotEpoch() to fire back-to-back within the exact same fraction of a millisecond. Because the StakingVault relies entirely on off-chain Keepers to call checkpointUser() to record time before a snapshot occurs, the autoFinalizeEpochs() loop gives the Keepers exactly zero physical block space to execute.
The Staking Vault instantly locks epochSnapshotted[epochId] = true for Epochs 1, 2, and 3 instantaneously. Any person who passively staked their tokens during those 3 weeks but failed to manually checkpoint themselves has their timeline permanently evicted by the _addContribution snapshot guard.

Result: All passive user staking yield for the skipped timeframe is mathematically erased from the protocol.

Status: Resolved
commit 177c18e

High Severity Issues

Issue #6: `createVault` Economic Gate Bypass via Flash-Loaned Self-Sandwiching

Severity: HIGH

Description

The `PartnerVaultFactory.createVault` function requires a minimum input of 500 USDC. This is intended to be a "burn" or "commitment" where the USDC is converted to PSRE and locked in the vault to ensure the partner has a vested interest in the ecosystem's success.

However, because the function allows the user to specify the `minPsreOut` (slippage) and the `fee` tier, an attacker can execute a Flash-Loaned Self-Sandwich. By using a flash loan to manipulate the pool's price immediately before the vault creation, the attacker ensures the Factory buys PSRE at an extremely inflated price. The attacker then sells their PSRE immediately after, capturing the Factory's 500 USDC and using it to repay the flash loan.

This reduces the cost of creating a "legitimate" looking `PartnerVault` from $500 to ~$5.00 (the cost of gas and DEX fees).

Affected Code

User-Controlled Slippage and Fee in `PartnerVaultFactory.sol`

```
None
// contracts/core/PartnerVaultFactory.sol:211
function createVault(
    uint256 usdcAmountIn,
    uint256 minPsreOut, // <--- Maliciously set to 1
    uint256 deadline,
    uint24  fee         // <--- Matches the pool being manipulated
) external nonReentrant returns (address vault) {
    require(usdcAmountIn >= S_MIN, "Factory: below S_MIN");

    // The swap is performed using user-supplied, unvalidated
slippage
    IFactorySwapRouter(router).exactInputSingle(
```

```
        IFactorySwapRouter.ExactInputSingleParams({
            // ...
            amountOutMinimum: minPsreOut, // <--- 1 allows massive
price impact
            fee: fee,
            // ...
        })
    );
}
```

Proof of Concept

The entire attack is executed in a single atomic transaction via a malicious contract:

1. **Flash Loan:** Attacker borrows 100,000 USDC from a provider (e.g., Aave or a Uniswap V3 Flash Swap).
2. **Front-run (Pump):** Attacker swaps the 100,000 USDC for PSRE in the target pool. This dramatically increases the price of PSRE (e.g., from 1 USDC to 100 USDC).
3. **The Victim (`createVault`):** The attacker calls `createVault` using 500 USDC (from the flash loan).
    - The Factory converts 500 USDC to PSRE at the inflated 100 USDC price.
    - The vault receives only 5 PSRE.
    - The 500 USDC is added to the pool's liquidity.
4. **Back-run (Dump):** Attacker sells all the PSRE they acquired in Step 2 back to the pool.
    - Because the pool now contains the extra 500 USDC from the Factory, the attacker receives their original 100,000 USDC plus the 500 USDC from the Factory (minus fees).
5. **Repayment:** Attacker repays the 100,000 USDC flash loan.
6. **Outcome:** The attacker keeps the ~500 USDC "profit," effectively refunding their vault creation. They now own a registered `PartnerVault` for the cost of transaction fees.

Impact

This completely breaks the economic security of the protocol.

- Zero-Cost Sybil Attack: An attacker can fill all 200 partner slots for the price of gas, creating a Denial of Service (DoS) for legitimate partners.

- Incentive Manipulation: Since the vault was created with almost zero PSRE (due to the 100 USDC/PSRE price), the `initialCumS` is very low. This makes it trivial for the partner to show "growth" later and earn disproportionate rewards from the `RewardEngine`.
- Total Loss of Gatekeeping: The S_{MIN} barrier is rendered purely cosmetic.

Recommendation

The protocol must remove the ability for the caller to dictate the slippage and price of the initial investment.

1. Remove `minPsreOut` as a User Input: The Factory should calculate the required minimum output internally using a trusted price source.
2. Use a Uniswap V3 TWAP Oracle: Before executing the swap, the Factory should check the current spot price against a 30-minute TWAP. If the deviation is higher than a small threshold (e.g., 2%), the transaction should revert. This makes sandwiching economically impossible.
3. Hardcode Standard Fee Tiers: Restrict the `fee` parameter to standard, high-liquidity tiers (0.05%, 0.3%) where it is much more expensive to manipulate the price.
4. Enforce a Minimum PSRE Quantity: Instead of `require(psreOut > 0)`, the Factory should require that `psreOut` is at least 95% of the expected market value of 500 USDC.

Status: Resolved

commit d2d04ad

Issue #7: Cross-Epoch StakeTime Accumulation Leaks Into Wrong Epoch Total

Severity: HIGH

Description:

The `StakingVault._checkpointUser` function calculates elapsed staking time (`block.timestamp - lastUpdateTimestamp`) and adds it directly to `currentEpochTotalStakeTime`. However, it fails to evaluate whether that elapsed time crosses one or more numerical epoch boundaries.

```
function _checkpointUser(address user) internal {
       UserStake storage s = userStakes[user];
       uint256 epoch = currentEpochId();

       // If user crosses an epoch boundary since last checkpoint,
       // we need to finalize their stakeTime for the previous epoch first.
       // For simplicity in v1, we accumulate across the current epoch only.
       // The snapshot mechanism handles epoch boundaries.

       if (s.lastUpdateTimestamp > 0 && block.timestamp > s.lastUpdateTimestamp) {
           uint256 elapsed   = block.timestamp - s.lastUpdateTimestamp;
           uint256 totalStake = s.psreBalance + s.lpBalance;
           uint256 delta     = totalStake * elapsed;

           s.accStakeTime             += delta;
           currentEpochTotalStakeTime += delta;
       }

       s.lastUpdateTimestamp = block.timestamp;
       userLastSnapshotEpoch[user] = epoch;
   }
```

When a passive staker deposits funds and stays idle for multiple weeks, their accumulated time legally "builds up". If they finally trigger a checkpoint in a much later epoch (e.g., Epoch 5), the elapsed timestamp difference evaluates to 35 days. The transaction then blindly dumps all 35 days of multi-epoch chronological weight directly into the global `currentEpochTotalStakeTime` of Epoch 5.

While the `recordStakeTime()` function attempts to cap the user's personal individual payout ticket to 7 days (`maxEpochStakeTime`), it does not retroactively deduct the excess 28 days of weight from the underlying `currentEpochTotalStakeTime`. As a result, the user's personal claim is artificially capped, but the global epoch total remains irrevocably inflated by multiple epochs' worth of leaked time.

```
function recordStakeTime(uint256 epochId) external {
        require(currentEpochId() > epochId,                        "StakingVault: epoch not
ended");
        require(userStakeTimeByEpoch[epochId][msg.sender] == 0,   "StakingVault: already
recorded");
        _checkpointUser(msg.sender);

        // Cap stakeTime to the epoch window to prevent unbounded accStakeTime overflow.
        // An epoch spans [epochStart, epochEnd). We cap at the duration of one full epoch.
        // This prevents a staker calling recordStakeTime 52 epochs late from claiming 52x
rewards.
        uint256 epochEnd      = genesisTimestamp + (epochId + 1) * EPOCH_DURATION;
        uint256 epochStart    = genesisTimestamp + epochId * EPOCH_DURATION;
        uint256 epochDuration = epochEnd - epochStart; // always = EPOCH_DURATION

        UserStake storage s       = userStakes[msg.sender];
        uint256 totalStake        = s.psreBalance + s.lpBalance;
        uint256 maxEpochStakeTime = totalStake * epochDuration; // theoretical max for this
epoch

        // Recorded stakeTime = min(accStakeTime, maxEpochStakeTime).
        // accStakeTime may exceed one epoch if user never checkpointed; we cap it.
        uint256 st;
        if (s.accStakeTime < maxEpochStakeTime) {
            st = s.accStakeTime;
        } else {
            st = maxEpochStakeTime;
        }
        userStakeTimeByEpoch[epochId][msg.sender] = st;

        // Reset accumulator so next epoch starts fresh.
        // Without this, accStakeTime grows unboundedly across epochs.
        s.accStakeTime = 0;
    }
```

Proof of Concept:

1. **The Sleeping Whale:** User A stakes 10,000 PSRE on Day 1 of Epoch 0. They do not interact with the protocol for the next 35 days (missing Epochs 0, 1, 2, 3, and 4 entirely).
2. **The Missing Weight:** Because `_checkpointUser` is not called, User A's massive stake weight is never pushed to the `currentEpochTotalStakeTime` for those missed epochs. Therefore, the global `totalStakeTimeByEpoch` denominators for Epochs 0-4 are artificially understated.
3. **The Activation:** During Epoch 5, User A wakes up and calls `stakePSRE(1)`. This triggers `_checkpointUser()`.

4. **The Dump:** The contract evaluates the elapsed time as exactly 35 days. It calculates `delta = 10,000 * 35 days`. Because there is no boundary sorting logic, this massive chronological delta is pushed perfectly and completely into Epoch 5's active pool tracker.
5. **Impact Realized:** The `snapshotEpoch(5)` locks Epoch 5's denominator at a massive, artificial peak.

Impact:

This creates a devastating two-sided economic collapse for honest stakers:

- **Artificial Boosts:** Stakers active during Epochs 1-4 receive significantly bloated rewards because the sleeping whale's capital was mathematically absent from the snapshot denominators.
- **Massive Dilution:** Active stakers during Epoch 5 suffer severe reward dilution because the sleeping whale's 35-day accumulated weight artificially inflates Epoch 5's denominator, mathematically crushing the legitimate fractional payouts for all honest actors claiming that week.

Recommendation:

Epoch time tracking strictly requires implicit mapping to boundaries to prevent cross-contamination.

Immediate Refactor: Abandon the simple `accStakeTime` delta tracking mechanism. The `StakingVault` must transition to a standard Time-Weighted Average Balance (TWAB) tracking system or an implementation modeled on `Synthetix StakingRewards`. Standard staking mechanisms handle boundaries implicitly within a global rate aggregator, preventing any user from physically controlling or leaking delayed chronological snapshots into active pools.

Status: Resolved

commit 6a3dda8

Issue #8: `createVault` `S_MIN` Economic Gate Bypass via Unvalidated Fee Tier Routing

Severity: HIGH

Description:

The `PartnerVaultFactory.createVault` function is designed to impose a meaningful economic entry cost (**$500 USDC**) for creating a new `PartnerVault`. This requirement, enforced via the `S_MIN` constant, serves as the primary defense against Sybil attacks and ensures that partners have "skin in the game" before occupying a protocol slot.

The vulnerability stems from the function's reliance on user-supplied Uniswap V3 **fee tiers** without validating the pool's liquidity or price. An attacker can deploy a USDC/PSRE pool on a valid but empty Uniswap V3 fee tier (e.g., the 1% tier) and provide a negligible amount of liquidity at an artificially inflated price.

When `createVault` is called, the protocol swaps the 500 USDC into this attacker-controlled pool. The attacker, acting as the sole Liquidity Provider, receives the 500 USDC and provides a near-zero amount of PSRE to the vault. The attacker then removes their liquidity to recover the USDC. This allows the attacker to create "ghost" vaults for effectively **zero capital cost**, bypassing the intended economic barrier entirely.

Affected Code:

Unvalidated `fee` and `minPsreOut` in `PartnerVaultFactory.sol`

```
None
// contracts/core/PartnerVaultFactory.sol:211
function createVault(
    uint256 usdcAmountIn,
    uint256 minPsreOut,
    uint256 deadline,
    uint24  fee // <--- Caller-controlled and unvalidated
) external nonReentrant returns (address vault) {
    // ...
    require(usdcAmountIn >= S_MIN, "Factory: below S_MIN ($500 USDC)");
    require(minPsreOut > 0, "Factory: slippage protection required"); // <---
Minimal requirement
```

```
    // ...

    // Swap routed through the specified fee tier
    IFactorySwapRouter(router).exactInputSingle(
        IFactorySwapRouter.ExactInputSingleParams({
            // ...
            fee: fee,
            // ...
        })
    );

    // ...
    require(psreOut > 0, "Factory: zero psreOut from initial buy"); // <--- Only
requires > 0
}
```

Proof of Concept:

1. **Preparation**: The attacker identifies an unused Uniswap V3 fee tier for USDC/PSRE (e.g., 10000 bps). They initialize this pool with a predatory price (e.g., **1 PSRE = 1,000,000 USDC**).
2. **Execution**: The attacker calls `createVault(500e6, 1, deadline, 10000)`.
3. **The Bypass**:
    - 500 USDC is taken from the attacker and swapped in the 1% pool.
    - The attacker (as the pool's LP) receives the 500 USDC.
    - The vault receives a fraction of a PSRE token (satisfying `psreOut > 0`).
4. **Recovery**: The attacker removes their liquidity from the Uniswap pool, recovering the 500 USDC. The only cost incurred is the Uniswap pool creation fee and gas.
5. **Griefing**: The attacker automates this process to create 200 vaults.

Impact:

The economic barrier intended to prevent spam and ensure partner commitment is rendered ineffective. This enables low-cost griefing of the `maxPartners` cap and allows the creation of "ghost" vaults that occupy protocol slots without any real capital commitment.

Recommendation:

To prevent the bypass of the economic gate, the protocol should introduce a minimum required PSRE output for vault creation, rather than relying solely on the USDC input.

- **Enforce Minimum PSRE Output**: Instead of `require(psreOut > 0)`, implement a `MIN_PSRE_OUT` (e.g., a value equivalent to $500 USDC at a reasonable market price) to ensure a minimum amount of capital is actually committed to the ecosystem.
- **Restrict Fee Tiers**: Validate the `fee` parameter against a whitelist of standard, highly liquid fee tiers (e.g., 0.05%, 0.3%) to prevent routing through attacker-controlled, illiquid pools.
- **Use an Oracle**: Use a price oracle to verify that the `psreOut` received is consistent with the current market value of the `usdcAmountIn`.

Status: Resolved

commit d2d04ad

Post Fixes Issues

Issue #9: autoFinalizeEpochs scarcity × K can exceed PSRE MAX_MINT_PER_EPOCH → DoS on vault operations

Severity: HIGH

Description:

`_autoFinalizeCount (K)` is stamped once before the batch loop (up to `AUTO_FINALIZE_MAX_EPOCHS = 10`) and every `_finalizeSingleEpoch` call multiplies `E_scarcity` by it. `E0_MAX` equals PSRE's per-epoch mint cap by construction, so a compressed batch where both `E_demand` and the scaled `scarcityCeiling` exceed `MAX_MINT_PER_EPOCH` causes `psre.mintForEpoch` to revert and DoSes every vault operation that calls `autoFinalizeEpochs.`

`contracts/periphery/RewardEngine.sol:618-702`

None

```
// E_scarcity = E0 × (1 - x)^2 where x = T / S_EMISSION  (§5.7)
uint256 E_scarcity;
if (T >= S_EMISSION) {
    E_scarcity = 0;
} else {
    uint256 x    = (T * PRECISION) / S_EMISSION;
    uint256 omx  = PRECISION - x;
    E_scarcity   = (E0 * omx / PRECISION) * omx / PRECISION;
}

// B = min(E_demand, E_scarcity, remaining)
// _autoFinalizeCount is 0 when finalizeEpoch() is called directly →
multiplier 1×.
uint256 scarcityCeiling = E_scarcity * (_autoFinalizeCount > 0 ?
_autoFinalizeCount : 1);
uint256 B = _min3(E_demand, scarcityCeiling, remaining);
...
uint256 P          = P_partners + P_stakers;
uint256 mintAmount = P < remaining ? P : remaining;

if (mintAmount > 0) {
```

```
    require(!IPausableToken(address(psre)).paused(), "RE: PSRE transfers
paused");
    psre.mintForEpoch(address(this), mintAmount, epochId); // ← reverts when >
25_200e18
    T += mintAmount;
}
```

`contracts/core/PSRE.sol:50, 149`

```
None
uint256 public constant MAX_MINT_PER_EPOCH = 25_200e18; // 0.002 * 12,600,000
= E0_MAX
...
require(
    epochMinted[epochId] + amount <= MAX_MINT_PER_EPOCH,
    "PSRE: epoch mint cap exceeded"
);
```

Numerical check. `E0_MAX = S_EMISSION * 2 / 1000 = 25_200e18`. With `K = 10` and `E_scarcity ≈ E0_MAX` (early emission, low `T`), `scarcityCeiling ≈ 252_000e18 ≈ 10 × MAX_MINT_PER_EPOCH`. If accumulated growth makes `E_demand > 25_200e18` as well, `mintAmount ≈ scarcityCeiling → mintForEpoch` reverts.

`contracts/core/PartnerVault.sol:225 — buy()` unconditionally calls `IRewardEngine(rewardEngine).autoFinalizeEpochs()`. `PartnerVaultFactory.createVault` does the same. Once the revert state is reached, every partner buy and every new-vault creation reverts until governance manually drains the backlog one epoch at a time via `finalizeEpoch()` (where `_autoFinalizeCount == 0` keeps the multiplier at 1×).

Impact

Permissionless DoS of the central protocol path. Preconditions are ordinary (≥ 2 unfinalized epochs with enough partner activity to push `E_demand` above `MAX_MINT_PER_EPOCH`), recoverable only via privileged out-of-band calls. Griefer cost = one transaction with a stale epoch backlog. Indirect secondary harm: partners are unable to register/grow during the DoS window, reducing protocol throughput and potentially stranding USDC that partners intended to deploy.

Recommendation

Clamp the scaled scarcity ceiling against PSRE's per-epoch mint cap so the batched mint always fits into a single epoch's budget. The scaled ceiling's purpose (capturing K weeks of compressed demand on one epoch) is preserved up to the hard cap and no further.

```
None
-    uint256 scarcityCeiling = E_scarcity * (_autoFinalizeCount > 0 ?
_autoFinalizeCount : 1);
+    uint256 scaled = E_scarcity * (_autoFinalizeCount > 0 ? _autoFinalizeCount :
1);
+    uint256 perEpochCap = IPSRE(address(psre)).MAX_MINT_PER_EPOCH();
+    uint256 scarcityCeiling = scaled > perEpochCap ? perEpochCap : scaled;
     uint256 B = _min3(E_demand, scarcityCeiling, remaining);
```

Alternative: split the compressed mint across the `K` historical `epochIds` so each epoch's `epochMinted[e]` stays within its own 25,200e18 budget.

Status: Resolved

commit 4ca5bfc

Issue #10: StakingVault settlement breaks invariant , retroactive reward theft, reward loss, and over-payment

Severity: HIGH

Description:

StakingVault v3 was rewritten from the original `accStakeTime` accumulator model to a Synthetix-style `rewardPerToken` settlement model. The implementation maintains a `lastSettledEpoch` cursor per user that tracks which epoch they have been settled through. The core correctness invariant, stated at `StakingVault.sol:440-442,` is:

"Must be called BEFORE any balance change. At call time the user's stored balance equals their balance during all unsettled epochs (because they haven't changed it since last settlement)."

Two compounding defects violate this invariant:

(a) `lastSettledEpoch` never initialized to the current epoch for new users

The `UserStake` struct defaults all fields to zero:

None

```
// StakingVault.sol:114-118
struct UserStake {
    uint256 psreBalance;    // 0
    uint256 lpBalance;      // 0
    uint256 lastSettledEpoch; // 0  ← NOT initialized to currentEpochId()
}
```

When a brand-new user calls `stakePSRE()`, settlement runs before the balance change:

None

```
// StakingVault.sol:201-207
function stakePSRE(uint256 amount) external nonReentrant {
    require(amount > 0, "StakingVault: zero amount");
```

```
    _settleFinishedEpochs(msg.sender);            // ① settlement with
balance = 0
    IERC20(psre).safeTransferFrom(msg.sender, address(this), amount);
    userStakes[msg.sender].psreBalance += amount;  // ② balance set AFTER
settlement
    totalPSREStaked += amount;
    emit PSREStaked(msg.sender, amount);
}
```

(b) `MAX_SETTLE_EPOCHS = 52` gas cap advances cursor past epochs that cannot be revisited with the correct balance

```
None

// StakingVault.sol:451-479
function _settleFinishedEpochs(address user) internal {
    if (lastFinalizedEpoch == type(uint256).max) return;

    UserStake storage s = userStakes[user];
    uint256 startEpoch  = s.lastSettledEpoch;  // 0 for new user
    uint256 endEpoch    = lastFinalizedEpoch;  // e.g. 59

    if (startEpoch > endEpoch) return;

    // Gas cap.
    uint256 cap = startEpoch + MAX_SETTLE_EPOCHS - 1; // 0 + 52 - 1 = 51
    if (endEpoch > cap) endEpoch = cap;               // endEpoch capped to 51

    for (uint256 e = startEpoch; e <= endEpoch; e++) {
        if (!epochDistributed[e]) continue;
        uint256 psreRpt = epochPSRERewardPerToken[e];
        uint256 lpRpt   = epochLPRewardPerToken[e];
        if (psreRpt > 0 && s.psreBalance > 0) {
            pendingRewards[user] += (s.psreBalance * psreRpt) /
REWARD_PRECISION;
        }
        if (lpRpt > 0 && s.lpBalance > 0) {
            pendingRewards[user] += (s.lpBalance * lpRpt) / REWARD_PRECISION;
```

```
        }
    }

    s.lastSettledEpoch = endEpoch + 1; // ← advances past the cap even if
balance was 0 / changed
}
```

When `lastFinalizedEpoch > 51`, settlement only processes epochs `[startEpoch, startEpoch+51]`. The cursor then advances past epochs the loop did visit AND (in the new-user case) past epochs the loop visited with the wrong balance. Any subsequent interaction credits the user's current (different) balance against historical `rewardPerToken[e]`.

The developer's fix states:

"New staker cursor initializes to `currentEpochId()` not epoch 0 preventing O(N) iteration from genesis for new users"

This initialization was never implemented. A search for all writes to `lastSettledEpoch` in the contract shows only one location `StakingVault.sol:478` inside the settlement loop. No initialization in `stakePSRE()`, `stakeLP()`, or any entry point.

Three exploit paths

Path 1: New-staker retroactive theft (`test_POC1_newStakerStealsHistoricalRewards`):

1. Alice stakes 1,000 PSRE at genesis. 60 epochs are finalized with 100 PSRE reward each. Alice is the sole staker.
2. After all 60 epochs finalize, attacker (who has never interacted) calls `stakePSRE(1000e18)`.
3. `_settleFinishedEpochs(attacker)` runs with `psreBalance = 0`, settles epochs 0-51 with zero rewards, sets `lastSettledEpoch = 52`. Then the balance becomes 1,000e18.
4. Attacker calls `claimAll()`. Settlement resumes from epoch 52, now using `psreBalance = 1000e18`. Attacker receives 800 PSRE for epochs 52-59 - epochs they never staked during.

5. Alice claims all 60 epochs: 6,000 PSRE. Combined claims: 6,800 PSRE from a pool of 6,000. Contract insolvent by 800 PSRE.

```
None

=== POC #1: New Staker Retroactive Reward Theft ===
Attacker staked at epoch 60, never staked before.
Rewards stolen for epochs 52-59: 800 PSRE
Alice's legitimate rewards: 6000 PSRE
Attacker's stolen rewards: 800 PSRE
Total claimed (Alice + Attacker): 6800 PSRE
Total distributed to SV: 6000 PSRE
```

Path 2: Existing staker LOSES rewards on unstake (`test_POC2_existingStakerLosesRewardsOnUnstake`):

1. Alice stakes 1,000 PSRE at epoch 0. Epochs 0-10 finalize and Alice claims, settling through epoch 10 (`lastSettledEpoch = 11`).
2. Epochs 11-80 finalize without Alice interacting (70 unsettled epochs > 52 cap).
3. Alice calls `unstakePSRE(500e18)`. Settlement processes epochs 11-62 with `balance = 1000` (correct), advances `lastSettledEpoch` to 63, then balance drops to 500.
4. Alice calls `claimAll()`. Settlement processes epochs 63-80 with `balance = 500` but Alice had 1,000 during those epochs. Permanent loss for 18 epochs.
5. Alice receives 3,600 PSRE instead of 8,100 PSRE. 4,500 PSRE permanently lost.

```
None
=== POC #2: Existing Staker Loses Rewards on Unstake ===
Alice staked 1000 PSRE for all 81 epochs.
Expected rewards: 8100 PSRE
Actual rewards  : 3600 PSRE
Rewards LOST    : 4500 PSRE
```

Path 3: Existing staker GAINS extra rewards on additional stake (`test_POC3_existingStakerGainsExtraRewardsOnAdditionalStake`):

1. Same setup as Path 2 through epoch 80.

2. Alice calls `stakePSRE(9000e18)`. Settlement processes epochs 11-62 with `balance = 1000` (correct), then balance increases to 10,000.
3. Alice calls `claimAll()`. Settlement processes epochs 63-80 with `balance = 10000` but Alice had 1,000 during those epochs. Receives 10× correct reward for 18 epochs.
4. Alice receives 12,150 PSRE instead of 8,100 PSRE. 4,050 PSRE created out of thin air → insolvency.

```
None
=== POC #3: Existing Staker Gets Extra Rewards on Additional Stake ===
Alice had 1000 PSRE staked for epochs 0-80.
Correct rewards : 8100 PSRE
Actual rewards  : 12150 PSRE
Extra rewards   : 4050 PSRE
```

The true Synthetix model avoids this entirely by using a single cumulative `rewardPerTokenStored` accumulator. Settlement is always O(1) - a single subtraction - requiring no loop and no gas cap:

```
None
// True Synthetix model — O(1), no loop, no cap, no invariant violation
earned = balance * (rewardPerTokenStored - userRewardPerTokenPaid) / 1e18;
```

The Prospereum implementation stores per-epoch rewardPerToken values instead of a cumulative accumulator. This forces an O(N) loop over unsettled epochs, which requires the gas cap, which breaks the invariant.

Impact:

- **Retroactive reward theft:** Any user who stakes for the first time after 53+ finalized epochs (≈1 year at 7-day epochs) receives rewards for historical epochs they never participated in.
- **Permanent reward loss (Path 2):** Users who unstake after partial settlement permanently lose rewards proportional to the unstaked fraction for all remaining unsettled epochs.
- **Contract insolvency (Paths 1 & 3):** Total claims exceed total distributed rewards. Late claimers or unstakers experience `safeTransfer` reverts.

- **Guaranteed trigger:** After 53 epochs × 7 days = 371 days, every new staker triggers Path 1. Not an edge case.
- **Amplification:** Attacker can stake an arbitrarily large amount to maximize stolen/inflated rewards. 10,000 PSRE staked → 8,000 PSRE theft per cycle.
- **Organic harm:** Affects honest users who don't interact for >52 weeks and then adjust their position, not just deliberate attackers.
- **Worsens over time:** Epoch gap grows with protocol age, increasing the magnitude of the error per interaction.

Recommendation:

Preferred architectural fix: Replace the per-epoch `rewardPerToken` loop with a true Synthetix-style cumulative accumulator. Eliminates the settlement loop and the `MAX_SETTLE_EPOCHS` gas cap entirely:

Standard Synthetix `StakingRewards` approach. Ensures:

1. Settlement always O(1) regardless of epoch gap
2. No gas cap needed
3. Correctness invariant holds unconditionally
4. New users auto-initialized (their `paidRewardPerToken` starts at the current cumulative value)

Per-epoch `rewardPerToken` values can be retained alongside the cumulative accumulator for auditability without affecting settlement logic.

Alternative fixes (if dev wants to preserve per-epoch bookkeeping for engagement proofs):

- **Option A: Balance checkpoints.** Store `balanceAt[user][epoch]` via OZ `Checkpoints`. Settlement reads historical balance per epoch. Fixes root cause. Cost: extra SSTORE per stake/unstake.
- **Option B: First-stake-epoch marker.** On zero→non-zero balance transition, record `s.firstStakeEpoch = currentEpochId()`. Settlement skips `e < firstStakeEpoch`. Must reset on each zero→non-zero transition. Cheap but addresses only new-user path.
- **Option C: Block balance changes until caught up.** Require `lastSettledEpoch >= lastFinalizedEpoch` before any stake/unstake; force permissionless `checkpointUser` catch-up calls. Preserves semantics; harsher UX.

Apply the same initialization in `stakeLP()`. Closes Path 1 only; Paths 2 and 3 still reachable via existing stakers with >52 epoch gaps. Architectural fix (Synthetix accumulator) closes all three.

Status: Resolved

commit 4ca5bfc

Medium Severity Issues

Issue #11: PartnerVault.snapshotEpoch Compresses Multi-Epoch Growth Into Single Epoch, Causing Permanent Reward Loss via Cap

Severity: MEDIUM

Description:

The RewardEngine processes rewards on a strictly sequential, weekly epoch basis. When an epoch is finalized, the engine calls snapshotEpoch() on every registered PartnerVault to determine the growth in the high-water mark (cumS) for that specific time period.

However, the PartnerVault.snapshotEpoch() implementation is **not epoch-aware**. It performs a live scan of current ERC-20 balances and calculates growth (deltaCumS) by subtracting the lastEpochCumS recorded during the **previous call**.

The protocol includes an autoFinalizeEpochs() function to process pending epochs sequentially during vault interactions . However, because the PartnerVault has no historical record of its balance at past epoch boundaries, the very first iteration of a "catch-up" loop captures the **entire accumulated growth** of all elapsed weeks in a single deltaCumS value.

Because the RewardEngine applies a per-epoch scarcity ceiling (E_scarcity or E_0), the rewards for this single "compressed" epoch are capped at one week's maximum. All subsequent iterations in the catch-up loop will return a deltaCumS of **0** because the vault's high-water mark was already updated to the current peak in the first iteration.

Affected Code:

1. Non-epoch-aware snapshot in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:351-363
function snapshotEpoch()
    external override onlyRewardEngine
    returns (uint256 deltaCumS)
```

```
{
    _updateCumS();
    if (cumS > lastEpochCumS) {
        deltaCumS = cumS - lastEpochCumS; // Captures ALL growth since last call
    } else {
        deltaCumS = 0;
    }
    lastEpochCumS = cumS; // Updates mark to current peak
}
```

2. Per-epoch scarcity cap in `RewardEngine.sol`

```
None
// contracts/periphery/RewardEngine.sol:619
uint256 B = _min3(E_demand, E_scarcity, remaining);
// If E_demand is huge (due to compression), B is capped at E_scarcity (one
epoch's worth)
```

Proof of Concept:

1. **Growth phase:** A PartnerVault grows its ***CumS*** steadily over 3 epochs (Epoch 1, 2, and 3). Total growth = 300 PSRE through direct transfers to `CustomerVault` addresses . No finalization occurs during this time.
2. **Delay:** For some reason (e.g., ack of calls), RewardEngine.finalizeEpoch() is not called for these 3 epochs.
3. **Catch-up Phase:** In Epoch 4, an entity triggers `autoFinalizeEpochs()`.
    a. **Iteration 1 (Epoch 1)**: `snapshotEpoch()` scans current balances and sees **300 PSRE** of growth. It returns this entire sum for Epoch 1. The `RewardEngine` calculates rewards but caps them at the `E_scarcity` limit (e.g., 120 PSRE).
    b. **Iteration 2 (Epoch 2)**: The loop calls `snapshotEpoch()` again. Since `lastEpochCumS` was updated to the peak in the previous step, it returns **0**.
    c. **Iteration 3 (Epoch 3)**: Returns **0**.
4. **Result**: The partner receives rewards for ~120 PSRE of growth instead of 300 PSRE. The rewards for the remaining 180 PSRE of growth are **permanently destroyed**.

Impact:

This issue results in the **permanent loss of earned rewards** for partners. While `autoFinalizeEpochs` reduces the likelihood of long delays, any period of inactivity—whether due to network congestion, gas spikes, or partners simply not calling `buy()`—will cause reward compression. This unfairly penalizes partners whose ecosystems grow via passive deposits and creates a griefing vector where actors can intentionally delay finalization to "burn" a competitor's rewards via the scarcity cap.

Recommendation:

The snapshotEpoch mechanism must be made epoch-aware to ensure that growth is distributed across the correct time windows, regardless of when the function is called.

- Epoch-Aware Snapshots: Modify snapshotEpoch to accept an ***epochId*** parameter.
- Time-Based Windows: Instead of a simple subtraction from the last call, snapshotEpoch should calculate growth based on the time window of the requested ***epochId***.
- Historical Tracking: Since ***CumS*** is a ratchet, the PartnerVault would need to store the
- ***CumS*** value at the end of each epoch to allow for accurate backfilling.
- Alternative: Alternatively, the RewardEngine could allow $E_{scarcity}$ to scale linearly with the number of epochs being finalized in a single transaction:
  $B = min\ (E_{demand}, K\ .\ E_{scarcity}, ...)$ where is the number of epochs since the last finalization.

Note:
In the latest commit , autoFinalizeEpochs() is added which tries to lazily update finalize epochs and are triggered on CreateVault and buy.

Status: Resolved

commit 7c13690

Issue #12: Static `partnerOf` Mapping Leads to Reward Lockout and Unauthorized CV Deployment After Ownership Transfer

Severity: MEDIUM

Description:

The `PartnerVaultFactory` maintains a `partnerOf` mapping that links a `PartnerVault` address to its creator. This mapping is set once during vault creation and is never updated. However, `PartnerVault` supports a two-step ownership transfer process (`updateOwner` and `acceptOwnership`).

Because the factory's `partnerOf` mapping is not synchronized with the vault's current owner, any system component relying on the factory mapping for authorization will be using stale data after an ownership transfer. This affects both the `RewardEngine` (for claiming rewards) and the `PartnerVaultFactory` (for deploying customer vaults), leading to a scenario where the new owner is locked out of critical functions while the former owner retains unauthorized privileges.

Affected Code:

The static mapping in **`PartnerVaultFactory`:**

```
None
// contracts/core/PartnerVaultFactory.sol:267
partnerOf[vault]    = msg.sender;
```

The stale check in **`RewardEngine`:**

```
None
// contracts/periphery/RewardEngine.sol:725
require(factory.partnerOf(vault) == msg.sender, "RE: not vault owner");
```

The stale check in **`PartnerVaultFactory`:**

```
None
// contracts/core/PartnerVaultFactory.sol:294
require(partnerOf[partnerVault] == msg.sender, "Factory: not vault owner");
```

Ownership transfer in **`PartnerVault`** (only updates local state):

```
None
// contracts/core/PartnerVault.sol:414
owner        = pendingOwner;
pendingOwner = address(0);
```

Proof of Concept:

1. **Creation:** Partner A creates a vault (Vault V). `factory.partnerOf[V]` is set to `A`.
2. **Transfer:** Partner A transfers ownership of Vault V to Partner B. `V.owner` becomes `B`, but `factory.partnerOf[V]` remains `A`.
3. **Reward Lockout & Theft:**
    a. Partner B calls `RewardEngine.claimPartnerReward(V)`. The call reverts because `factory.partnerOf[V]` is not `B`.
    b. Partner A calls `RewardEngine.claimPartnerReward(V)`. The call succeeds, allowing A to drain rewards from a vault they no longer own.
4. **CV Deployment Lockout & Intrusion:**
    a. Partner B attempts to call `factory.deployCustomerVault(V, ...)`. The call reverts because `factory.partnerOf[V]` is not `B`.
    b. Partner A calls `factory.deployCustomerVault(V, ...)`. The call succeeds, allowing A to register unauthorized `CustomerVaults` in B's vault.

Impact:

This results in a complete breakdown of access control for vaults that have changed ownership. The new owner is effectively stripped of administrative rights regarding rewards and expansion, while the previous owner retains a "backdoor" to steal funds and manipulate the vault's ecosystem.

Recommendation:

Remove the reliance on the static `partnerOf` mapping for authorization. Instead, all components should query the current owner directly from the `PartnerVault` contract.

- In `RewardEngine.sol`, change the check to:
  `require(IPartnerVault(vault).owner() == msg.sender, "RE: not vault owner");`

- In `PartnerVaultFactory.sol`, change the check to:
  `require(IPartnerVault(partnerVault).owner() == msg.sender, "Factory: not vault owner");`

Status: Resolved

commit 44f17d8

Issue #13: recordStakeTime Cap Based on Current Balance Zeroes Rewards After Unstaking

Severity: MEDIUM

Description:

The `recordStakeTime` function in `StakingVault.sol` implements a cap on the amount of stake time a user can record for an epoch to prevent unbounded claims. However, this cap (`maxEpochStakeTime`) is calculated using the user's current balance at the moment the function is called, rather than the balance they held during the target epoch.

If a user unstakes their tokens before calling `recordStakeTime`, their current balance is 0, resulting in a `maxEpochStakeTime` of 0. Consequently, any legitimate stake time accumulated in `accStakeTime` during the epoch is silently zeroed out by the `min()` logic, and the user is permanently barred from claiming rewards for that epoch.

Affected Code:

1. Calculation of the cap using current balance

```
None
// contracts/periphery/StakingVault.sol:245-247
UserStake storage s       = userStakes[msg.sender];
uint256 totalStake        = s.psreBalance + s.lpBalance;
uint256 maxEpochStakeTime = totalStake * epochDuration;
```

2. The zeroing logic in **`recordStakeTime`**

```
None
// contracts/periphery/StakingVault.sol:252-257
if (s.accStakeTime < maxEpochStakeTime) {
    st = s.accStakeTime;
} else {
    st = maxEpochStakeTime;
}
userStakeTimeByEpoch[epochId][msg.sender] = st;
```

3. Balance reduction in **`unstakePSRE`**

```
None
// contracts/periphery/StakingVault.sol:162-163
_checkpointUser(msg.sender);
userStakes[msg.sender].psreBalance -= amount;
```

Proof of Concept:

1. **Staking:** User A stakes 1000 PSRE at the beginning of Epoch N.
2. **Accumulation:** Throughout Epoch N, `_checkpointUser` is triggered, and `s.accStakeTime` correctly accumulates **1000 * epochDuration**.
3. **Epoch End**: The epoch ends, and the `RewardEngine` calls `snapshotEpoch(N)`. The user's contribution is included in the global `totalStakeTimeByEpoch[N].`
4. **Unstaking:** Before calling `recordStakeTime`, User A calls `unstakePSRE(1000)` to recover their liquidity. Their `psreBalance` becomes 0.
5. **Recording:** User A calls `recordStakeTime(N)`.
    - `totalStake` is now 0.
    - `maxEpochStakeTime` is calculated as **0 * epochDuration = 0**.
    - `st` is determined as `min(accStakeTime, 0)`, which is 0.
    - `userStakeTimeByEpoch[N][User A]` is set to 0.
6. **Result:** User A has contributed a full epoch of stake to the protocol but receives 0 rewards.

Impact:

This vulnerability causes legitimate stakers to lose their rewards if they unstake before recording their stake time. Because the "already recorded" guard prevents a second call to `recordStakeTime`, this loss is permanent. Furthermore, because the user's stake time was already captured in the global `totalStakeTimeByEpoch` during the snapshot, their "lost" reward share is effectively redistributed to other stakers in that epoch rather than being returned to the treasury or handled as unclaimed funds.

Recommendation:

The cap should not be based on the balance at the time of the claim. Instead, the protocol should rely on the `accStakeTime` accumulator, which is already updated based on the balance held at the time of the checkpoint.

If a cap is strictly required to prevent "unbounded" claims, it should be based on the maximum balance held during the epoch or a constant based on the known duration:

1. **Remove the balance-based cap:** Since `accStakeTime` is updated via checkpoints, it is inherently bounded by the user's actual staking history.
2. **Use Historical Snapshots:** Ensure that `recordStakeTime` uses a snapshot of the balance from the end of the target epoch.
3. Before User Unstakes , perform a call to recordStakeTime.

Status: Resolved

commit 6a3dda8

Issue #14: CustomerVault.reclaimUnclaimed is Permanently Unreachable Due to Missing Caller in PartnerVault

Severity: MEDIUM

Description:

The `CustomerVault` contract provides a function called `reclaimUnclaimed()`, designed to allow the parent `PartnerVault` to recover funds that were distributed to a customer but never claimed. This function is protected by the `onlyParent` modifier, which strictly requires the caller to be the `parentVault` (the address of the `PartnerVault` contract).

However, a review of the `PartnerVault` contract reveals that it contains no function or internal logic that makes an external call to `reclaimUnclaimed()`. Because the modifier prevents any address other than the `PartnerVault` contract itself from calling the function, and the `PartnerVault` contract never calls it, the function is effectively "dead code."

Affected Code:
`CustomerVault.reclaimUnclaimed` and **`onlyParent`** modifier:

```
None
// contracts/core/CustomerVault.sol
modifier onlyParent() {
    require(msg.sender == parentVault, "CustomerVault: only parent vault");
    _;
}

function reclaimUnclaimed() external onlyParent {
    // Logic to return unclaimed PSRE to the parent vault
    // ...
}
```

`PartnerVault` Missing Integration: Review of `PartnerVault.sol` shows that all functions (`buy, distributeToCustomer, transferOut, registerCustomerVault, etc.`) manage state or transfer tokens, but none ever invoke `reclaimUnclaimed()` on any associated `CustomerVault` addresses.

Impact:

Funds distributed to inactive customers are effectively locked in the `CustomerVault` clones. While a workaround exists, it forces the partner to absorb an economic penalty (reduction in `ecosystemBalance`) that the system was specifically designed to avoid via the `reclaimUnclaimed` mechanism.

Recommendation:

Implement a recovery function within the `PartnerVault` contract that allows the owner to trigger the reclaim process.

Proposed addition to **`PartnerVault.sol`:**

```
None
function reclaimFromCV(address customerVault) external onlyOwner {
    require(registeredCustomerVaults[customerVault], "PartnerVault: CV not
registered");
    ICustomerVault(customerVault).reclaimUnclaimed();
}
```

This ensures that the `msg.sender` of the `reclaimUnclaimed` call is the `PartnerVault` contract itself, satisfying the onlyParent requirement and allowing funds to return to the ecosystem without triggering leakage penalties.

Status: Resolved

commit 44f17d8

Issue #15: `customerVaultList` Grows Monotonically Without Deregistration, Permanent Registration Cap DoS

Severity: MEDIUM

Description:

The `PartnerVault` contract maintains a registry of `CustomerVault` (CV) addresses used to track the overall ecosystem balance. When a new CV is registered via `registerCustomerVault()`, it is pushed onto the `customerVaultList` array. This process is guarded by a hard limit set by `MAX_CUSTOMER_VAULTS` (1,000 entries).

However, there is no mechanism provided within the contract to remove a CV from the `customerVaultList` or to deregister a vault. Because the array only grows and never shrinks, once a partner has registered 1,000 customer vaults, regardless of whether those vaults are now empty, abandoned, or depleted, the `require` check permanently blocks the registration of any new customer vaults.

Affected Code:

1. The monotonically growing registration process

```
None
// contracts/core/PartnerVault.sol:323-338
function registerCustomerVault(address customerVault) external {
    require(msg.sender == owner || msg.sender == factory, "PartnerVault: not
owner");
    require(customerVault != address(0),              "PartnerVault: zero CV
address");
    require(!registeredCustomerVaults[customerVault], "PartnerVault: CV already
registered");
    require(customerVaultList.length < MAX_CUSTOMER_VAULTS, "PartnerVault: max CVs
reached"); // Permanent cap

    // ... validation logic ...

    registeredCustomerVaults[customerVault] = true;
    customerVaultList.push(customerVault); // Array grows monotonically
    emit CustomerVaultRegistered(address(this), customerVault);
}
```

2. Gas waste in `_updateCumS` due to lack of cleanup

```
None
// contracts/core/PartnerVault.sol:385-387
for (uint256 i = 0; i < n; i++) {
    totalEcosystem += IERC20(psre).balanceOf(customerVaultList[i]);
}
```

Proof of Concept:

1. Operation: A Partner creates and registers `CustomerVaults` for customers as they join the ecosystem.
2. Attrition: Over time, many customers leave the ecosystem; their vaults are drained and become inactive.
3. Cap Reach: After 1,000 total registrations (including the inactive ones), the `customerVaultList.length` reaches `MAX_CUSTOMER_VAULTS`.
4. DoS: The partner attempts to onboard a 1,001st customer and calls `registerCustomerVault()`.
5. Failure: The transaction reverts with "PartnerVault: max CVs reached".
6. Lock-out: Because there is no `deregister` function, the partner is permanently unable to add new customers to the current vault. To resume business, the partner must deploy a new `PartnerVault`, which resets their `cumS` history and eliminates their reward eligibility progress.

Impact:

This is a "slow-burn" Denial of Service. While not an immediate exploit, it creates a hard ceiling on the scalability of a partner's business. Long-term partners are eventually forced into a position where they must choose between a frozen customer base or losing all historical progress by migrating to a new vault. Additionally, it causes unnecessary gas consumption during `snapshotEpoch` as the contract iterates over hundreds of potentially empty vaults.

Recommendation:

Implement a mechanism to deregister `CustomerVaults` and remove them from the `customerVaultList`.

- **Add a `deregisterCustomerVault` function**: Allow the owner to remove a CV.
- **Use the "swap-and-pop" pattern**: To remove the element from `customerVaultList` in O(1) time to avoid expensive array shifting.

- **Update State**: Set `registeredCustomerVaults[cv] = false`.
- **Optional Optimization**: In `_updateCumS`, consider adding a check to skip vaults that have been marked as inactive or are known to be empty, although a proper deregistration function is the primary fix for the DoS.

Status: Resolved

commit 7c13690

Issue #16: `allVaults` Array Is Append-Only With No Removal Path — Permanent Slot Exhaustion and Gas Ratchet

Severity: MEDIUM

Description:

The `PartnerVaultFactory` maintains a global registry of all deployed vaults in the `allVaults` array. This array is currently used for two primary purposes: as a capacity guard to limit the total number of partners (`maxPartners`) and as the iteration source for the `RewardEngine` during epoch finalization.

However, the `allVaults` array is append-only. Once a vault is added via `createVault()`, it can never be removed. This creates a critical scaling issue: inactive or abandoned vaults permanently occupy a slot in the factory's capacity. Once the `maxPartners` limit is reached, the protocol will permanently block the creation of new vaults, even if the majority of existing vaults are defunct.

Additionally, because `RewardEngine.finalizeEpoch` iterates through the entire array and performs external calls for every registered vault, the gas cost of finalizing an epoch increases monotonically over time, regardless of the number of active participants.

Affected Code:

1. Permanent capacity consumption in `PartnerVaultFactory.sol`

```
None
// contracts/core/PartnerVaultFactory.sol:219
require(allVaults.length < maxPartners, "Factory: max partners reached");
// ...
// contracts/core/PartnerVaultFactory.sol:268
allVaults.push(vault);
```

2. Gas ratchet in `RewardEngine.sol`

```
None
// contracts/periphery/RewardEngine.sol:463
address[] memory vaults = factory.getAllVaults();
uint256 nVaults = vaults.length;
// ...
```

```
// contracts/periphery/RewardEngine.sol:483
for (uint256 i = 0; i < nVaults; i++) {
    address vault = vaults[i];
    // ...
    // Every vault, active or inactive, triggers these external calls
    IPartnerVault(vault).snapshotEpoch();
    uint256 currentCumS = IPartnerVault(vault).getCumS();
    // ...
}
```

Proof of Concept:

1. **Growth**: Over several months, 200 partners create `PartnerVaults`. `allVaults.length` now equals `maxPartners`.
2. **Attrition**: 150 of these partners stop using the protocol. Their vaults are marked as `vaultActive = false` by the `RewardEngine` after 52 epochs of inactivity.
3. **Blocked Entry**: A new partner attempts to join by calling `createVault()`. Even though only 50 vaults are active, the transaction reverts with **"Factory: max partners reached"**.
4. **Gas Inflation**: Each call to `finalizeEpoch()` must still iterate through all 200 vault addresses and perform `snapshotEpoch()` and `getCumS()` calls, wasting gas on the 150 inactive vaults.

Impact:

- **Scalability Ceiling**: The protocol has a hard, non-recoverable limit on the total number of partners it can ever support. It cannot recycle slots from dead vaults.
- **Economic Penalty**: The cost of maintaining the protocol (gas for finalization) increases linearly with the historical number of vaults, not the current number of active users. This creates a "gas ratchet" that could eventually make epoch finalization prohibitively expensive or exceed block gas limits if `maxPartners` is increased.

Recommendation:

Implement a way to prune the `allVaults` registry and optimize the iteration process:

1. **Vault Removal**: Provide a mechanism to remove vaults from `allVaults` and the `partnerOf` mapping when a vault is decommissioned or becomes permanently inactive. Use the "swap-and-pop" method to ensure $O(1)$ removal.

2. **Activity-Based Iteration**: In `RewardEngine.finalizeEpoch`, ensure that the logic skips external calls to vaults that are marked as inactive. While this doesn't fix the array length, it mitigates the gas ratchet.
3. **Dynamic Capacity**: Instead of using `allVaults.length` as the capacity guard, maintain a separate `activeVaultCount` tracker that is decremented when a vault is removed.

Status: Resolved

commit 7c13690

Issue #17: LP Tokens Weighted 1:1 With PSRE in StakeTime Computation Enables Cheap Reward Dilution

Severity: MEDIUM

Description:

The `StakingVault` contract allows users to stake both PSRE tokens and LP tokens. The total stake time accumulated by a user is calculated in the `_checkpointUser` function by simply summing the balances of these two different assets:

```
totalStake = s.psreBalance + s.lpBalance
```

The vulnerability lies in the assumption that one unit of LP token has the same economic value as one unit of PSRE. In practice, LP tokens are shares of a liquidity pool; their unit price is determined by the total liquidity and the total supply of LP tokens. If the LP token price is significantly lower than the PSRE token price (which is highly likely), an attacker can acquire a massive quantity of LP tokens for a fraction of the cost of an equivalent amount of PSRE.

Because the system treats them **1:1**, the attacker can accrue `stakeTime` at the same rate as a pure PSRE staker, effectively capturing a disproportionate share of the epoch's staker reward pool while committing far less capital.

Affected Code:

1. 1:1 Weighting in `_checkpointUser`

```
None
// contracts/periphery/StakingVault.sol:312
uint256 totalStake = s.psreBalance + s.lpBalance;
uint256 delta     = totalStake * elapsed;
```

2. Explicit confirmation in `stakeLP` NatSpec

```
None
// contracts/periphery/StakingVault.sol:173-174
/** * @notice Stake LP tokens. Treated equivalently to PSRE for stakeTime
accounting.
 * stakeTime = lpAmount × duration (no weighting multiplier per spec §2.4).
```

```
*/
```

Proof of Concept:

1. **Market Discrepancy**: Assume **1 PSRE** is worth **10 USDC**, while **1 LP token** (due to the total supply of the pool) is worth **0.1 USDC**.
2. **Acquisition**: An attacker spends **1,000 USDC**.
   - **Buying PSRE**: They would get **100 PSRE**.
   - **Buying LP**: They get **10,000 LP tokens**.
3. **Staking**: The attacker stakes **10,000 LP tokens** via `stakeLP()`.
4. **Accumulation**: During the epoch, the attacker's `totalStake` is **10,000**. A legitimate staker who spent **1,000 USDC** on PSRE has a `totalStake` of only **100**.
5. **Reward Claim**: Both call `recordStakeTime()` and `claimStake()`.
6. **Outcome**: The attacker receives **100x** more rewards than the PSRE staker, despite both having committed the same **1,000 USDC** of value.

Impact:

This is an economic exploit that leads to systemic reward dilution. Pure PSRE stakers are penalized for holding the more valuable asset, while "mercenary" stakers can manipulate the reward distribution by shifting into the cheapest available asset (LP tokens) to inflate their stake time. This undermines the fairness of the "Proof of Prosperity" model.

Recommendation:

Introduce a weighting multiplier or a value-normalization mechanism to ensure that `stakeTime` represents actual economic value rather than raw token counts. Revise the protocol specification (2.4) to move away from the 1:1 weighting and adopt a value-based approach.

Status: Resolved

commit 6a3dda8

Issue #18: transferOut Guard Bypassed via CustomerVault Destinations Enabling Cross-Vault cumS Inflation

Severity: MEDIUM

Description:

The `PartnerVault.transferOut` function is designed to allow partners to move PSRE out of their ecosystem. To prevent the artificial inflation of rewards, it includes a guard to ensure that funds are not transferred directly to another registered vault: `require(IPartnerVaultFactory(factory).isRegisteredVault(to) == false, "PartnerVault: cannot transferOut to registered vault")`.

However, this guard is insufficient. The `isRegisteredVault` function in `PartnerVaultFactory` only checks the `partnerOf` mapping, which exclusively contains addresses of `PartnerVault` contracts. It does **not** check if the destination is a `CustomerVault` (CV).

Since `CustomerVault` addresses are not stored in the `partnerOf` mapping, a user can call `transferOut` with a `CustomerVault` address as the destination. This allows PSRE to be routed from **PartnerVault A** to a **CustomerVault** belonging to **PartnerVault B**. Because **PartnerVault B** calculates its **cumS** (high-water mark) by scanning the actual balances of its associated **CustomerVaults** in `_updateCumS()`, this transfer artificially inflates the **cumS** of Vault B and generates rewards, while the **cumS** of Vault A remains at its prior high-water mark due to the ratchet mechanism.

Affected Code:

1. The insufficient guard in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:270-271
require(
    IPartnerVaultFactory(factory).isRegisteredVault(to) == false,
    "PartnerVault: cannot transferOut to registered vault"
);
```

2. The narrow check in `PartnerVaultFactory.sol`

```
None
// contracts/core/PartnerVaultFactory.sol:325
function isRegisteredVault(address vault) external view override returns (bool) {
    return partnerOf[vault] != address(0); // Only checks PartnerVaults, ignores
CustomerVaults
}
```

3. The balance scan that captures the routed funds in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:385-387
for (uint256 i = 0; i < n; i++) {
    totalEcosystem += IERC20(psre).balanceOf(customerVaultList[i]);
}
// ...
if (ecosystemBalance > cumS) {
    cumS = ecosystemBalance; // Ratchets up based on routed funds
}
```

Proof of Concept:

1. **Setup**: Attacker controls **PartnerVault A** (PV_A) and **PartnerVault B** (PV_B).
2. **Target**: Attacker identifies a **CustomerVault** (CV_B) registered to PV_B.
3. **Routing**: Attacker calls `PV_A.transferOut(CV_B, X)`.
   - `to` (CV_B) is not in `partnerOf`, so `isRegisteredVault(CV_B)` returns **false**.
   - The guard passes, and **X** PSRE is transferred from PV_A to CV_B.
4. **Inflation**: During the next epoch finalization, `PV_B.snapshotEpoch()` -> `PV_B._updateCumS()` is called.
   - The scan detects **X** PSRE in CV_B.
   - `PV_B.cumS` ratchets up by **X**.
5. **Reward Extraction**: `RewardEngine` sees the growth in `PV_B.cumS` and allocates rewards to the owner of PV_B.
6. **Cycling**: The attacker (or a colluding customer) withdraws **X** from CV_B. `PV_B.ecosystemBalance` decreases, but `PV_B.cumS` remains inflated.
7. **Systemic Gain**: The same **X** PSRE has now contributed to the **cumS** of two different vaults (A and B) without any new PSRE entering the system, effectively duplicating the reward-generating power of the tokens.

Impact:

This vulnerability allows for the artificial inflation of the cumS metric across multiple vaults. It enables partners to "game" the reward system by routing existing PSRE between vaults to trigger rewards that should only be earned through genuine ecosystem growth or new capital inflows. This dilutes the rewards for legitimate partners and violates the protocol's core economic invariant that cumS should represent actual growth.

Recommendation:

Update the `transferOut` guards to ensure that funds cannot be sent to any vault-related address, including **CustomerVaults**.

Proposed Change in **`PartnerVault.sol`**:

```
None
// Add a check to verify the destination is not a CustomerVault of any partner
require(
    IPartnerVaultFactory(factory).isCustomerVaultOf(to) == address(0),
    "PartnerVault: cannot transferOut to a customer vault"
);
```

This ensures that PSRE exiting a vault must go to an external wallet, preventing the routing of funds between different partner ecosystems to inflate high-water marks.

Status: Resolved

commit 44f17d8

Issue #19: Non-Recording Staker Checkpoints Inflate `totalStakeTime` Denominator, Diluting Rewards and Locking Minted PSRE

Severity: MEDIUM

Description:

The `StakingVault` contract tracks stake time for rewards using a global accumulator (`currentEpochTotalStakeTime`) and individual user records (`userStakeTimeByEpoch`).

The vulnerability exists because the global accumulator is updated unconditionally, while the individual records are updated via an opt-in mechanism. Whenever a user stakes or unstakes, `_checkpointUser` is triggered, which immediately adds the user's accumulated stake time to `currentEpochTotalStakeTime`. However, a user's contribution to the reward pool is only formalized in `userStakeTimeByEpoch` if the user explicitly calls the `recordStakeTime` function.

If a user triggers a checkpoint (by staking or unstaking) but never calls `recordStakeTime`, their stake time remains part of the global `totalStakeTimeByEpoch` (the denominator) but is absent from the individual numerators. This results in a mathematical discrepancy where the sum of all claimable rewards is less than the total amount minted for that epoch.

Affected Code:

1. Unconditional global update in `_checkpointUser`

```
None
// contracts/periphery/StakingVault.sol:315-316
s.accStakeTime             += delta;
currentEpochTotalStakeTime += delta; // Globally added regardless of recording
intent
```

2. Opt-in individual recording in `recordStakeTime`

```
None
// contracts/periphery/StakingVault.sol:257
userStakeTimeByEpoch[epochId][msg.sender] = st; // Only called upon explicit
request
```

3. Reward calculation using the inflated denominator

```
None
// contracts/periphery/RewardEngine.sol:753
uint256 reward = (epochStakersPool[epochId] * userST) / totalST;
// totalST (totalStakeTimeByEpoch) includes non-recording users
```

Proof of Concept:

1. **Staking**: User A stakes **1,000 PSRE**. User B stakes **1,000 PSRE**.
2. **Interaction**: Both users trigger `_checkpointUser` at the end of the epoch. `currentEpochTotalStakeTime` now includes both users' contributions (e.g., $2,000 \times duration$).
3. **Snapshot**: `snapshotEpoch` is called, and `totalStakeTimeByEpoch` is set to $2,000 \times duration$.
4. **Recording**:
   - User A calls `recordStakeTime()`.
   - User B **does not** call `recordStakeTime()`.
5. **Claiming**: User A calls `claimStake()`. Their reward is calculated as:

$$Reward = \frac{Pool \times 1,000 \times duration}{2,000 \times duration} = 50\% \text{ of the pool}$$

6. **Result:** If User B had recorded, User A would have received a share relative to B. Because B didn't record, A's reward is diluted by B's presence in the denominator. The other 50% of the minted rewards for that epoch are permanently locked in the `RewardEngine` with no way to be claimed or recovered.

Impact:

- **Reward Dilution**: Legitimate stakers who follow the protocol and record their time receive fewer rewards than they should, as their share is diluted by users who trigger checkpoints but don't record.
- **Permanent Fund Lockup**: A significant portion of the PSRE emission budget can be minted into the `RewardEngine` and then permanently locked because the corresponding numerators (individual `userStakeTimeByEpoch`) were never created.

- **Griefing Vector**: A malicious actor can stake large amounts and trigger checkpoints to inflate the denominator, effectively stealing reward value from other stakers and burning the protocol's emission budget.

Recommendation:

Ensure that the global accumulator only includes stake time that is actually eligible for recording, or implement a mechanism to reconcile the global total with recorded totals.

1. **Shift the Global Update**: Instead of updating `currentEpochTotalStakeTime` in `_checkpointUser`, update it within `recordStakeTime` at the moment the user commits their time to the epoch.
2. **Post-Epoch Reconciliation**: In `RewardEngine.finalizeEpoch`, calculate the actual total $\sum userStakeTimeByEpoch$ and only mint the amount corresponding to that sum, rather than minting a fixed pool based on the raw `totalStakeTime`.
3. **Automatic Recording**: Consider making `recordStakeTime` internal and calling it automatically during `_checkpointUser` or `snapshotEpoch` to ensure all contributions are recorded.

Status: Resolved

commit 6a3dda8

Issue #20: Independent PSRE and RewardEngine Pause States Create "Ghost" Emission Consumption

Severity: MEDIUM

Description:

The protocol employs two separate pausing mechanisms: one in the `PSRE` token contract (governed by `PAUSER_ROLE`) and one in the `RewardEngine` contract (governed by `onlyOwner`). These two states are completely decoupled, creating a critical asymmetry during emergency scenarios.

The `PSRE` token's `_update` function is designed to block all transfers when paused, but it explicitly exempts minting operations (where `from == address(0)`). Meanwhile, the `RewardEngine.finalizeEpoch` function—which triggers the minting of new PSRE—is only gated by the `RewardEngine`'s own pause state.

If the `PSRE` token is paused but the `RewardEngine` remains unpaused, the system enters a state where the **emission budget is consumed**, but the **rewards are trapped**. `finalizeEpoch` will continue to execute, minting new PSRE and incrementing the total supply (T), effectively "using up" the protocol's finite emission reserve ($S_{emission} - T$). However, because all outgoing transfers (claims, unstaking, distributions) are blocked by the `PSRE` pause, no one can actually receive these tokens.

Affected Code:

1. PSRE: Minting is allowed while transfers are paused

```
None
// contracts/core/PSRE.sol:184-186
function _update(address from, address to, uint256 value) internal override {
    if (from != address(0)) { // Minting (from == address(0)) skips this check
        require(!paused(), "PSRE: transfers paused");
    }
    super._update(from, to, value);
}
```

2. RewardEngine: Finalization is independent of PSRE pause state

```
None
// contracts/periphery/RewardEngine.sol:442
function finalizeEpoch(uint256 epochId) external nonReentrant whenNotPaused {
    // ...
    // This call succeeds even if PSRE is paused because it's a mint
    psre.mint(address(this), mintAmount);
    T += mintAmount;
    // ...
}
```

3. RewardEngine: Claims fail when PSRE is paused

```
None
// contracts/periphery/RewardEngine.sol:732
// This triggers PSRE._update with from != address(0), which reverts if PSRE is
paused.
IERC20(address(psre)).safeTransfer(msg.sender, owed);
```

Proof of Concept:

1. **Emergency Pause**: An emergency is detected (e.g., a vulnerability in a peripheral contract). The `PAUSER_ROLE` holder calls `PSRE.pause()`.
2. **Emission Continues**: The `RewardEngine` remains unpaused. A user or bot calls `RewardEngine.finalizeEpoch()`.
3. **Budget Consumption**: The `RewardEngine` mints new PSRE to itself and increases the global T (cumulative minted supply). This reduces the remaining emission budget ($S_{emission} - T$).
4. **Locked Rewards**: Partners try to call `claimPartnerReward()` or stakers try to call `claimStake()`. These transactions revert because `safeTransfer` triggers the `PSRE` pause check.
5. **Outcome**: The protocol has "spent" its emission for that epoch, but the tokens are stuck in the `RewardEngine`. The reward budget for that epoch is permanently consumed without any user actually receiving the funds.

Impact:

This creates a "ghost emission" scenario. In a prolonged emergency where `PSRE` is paused but `RewardEngine` is not, the finite emission reserve is drained without benefiting the users. This permanently reduces the total rewards available for the lifetime of the protocol

and unfairly penalizes users who should have earned rewards during that period but cannot claim them.

Recommendation:

Synchronize the pause states or ensure that the `RewardEngine` is aware of the `PSRE` token's status.

- **Linked Pause**: Update `RewardEngine.finalizeEpoch` to check the pause state of the `PSRE` token before minting: `solidity` `require(!IPSRE(address(psre)).paused(), "RE: PSRE token is paused");`
- **Unified Pause**: Move the pause authority to a single coordinator contract that manages the pause state for both the token and the engine simultaneously.
- **Safe Minting**: Only allow `finalizeEpoch` to proceed if the distribution path (the ability to transfer tokens) is also open.

Ah, the classic "burst vs. steady" math trap. It's a subtle one, but essentially, the protocol is accidentally rewarding procrastination (or network congestion) by letting a single massive growth spike skip the line on the EMA decay.

Status: Resolved

commit 303cd2f

Issue #21: Compressed Multi-Epoch Δ effectiveCumS Spikes EMA Score, Granting Undeserved Tier Promotion

Severity: **MEDIUM**

Description:

The `RewardEngine` uses an Exponential Moving Average (EMA) to track a vault's performance and assign it a reward tier (Bronze, Silver, or Gold). The EMA is updated every epoch using the following formula:

$$R_{new} = (R_{old} \cdot (1 - \theta)) + (\Delta effectiveCumS \cdot \theta)$$

Where *0 ~ 1/13*

A critical logic error exists in how `PartnerVault.snapshotEpoch()` interacts with this EMA. The `snapshotEpoch` function is **not epoch-aware**; it simply returns the total growth since the last time it was called.

If `finalizeEpoch` is delayed for "K" epochs, the first call to `snapshotEpoch` will return the sum of growth for all "K" epochs. In the EMA formula, a single massive spike of

Delta = K . x (where x is average growth) produces a **significantly higher** "R" value than "K" individual updates of "x". Because the EMA decays slowly, this artificial spike pushes the vault into a higher tier prematurely, granting it an unearned reward multiplier (1.2x vs. 0.8x) for many subsequent epochs.

Status: **Resolved**

commit 7c13690

Post Fixes Issues

Issue #22: autoFinalizeEpochs() Scaled Minting Collides with PSRE Epoch Mint Cap, Permanent Protocol DoS

Severity: MEDIUM

Description:

The developer's `_autoFinalizeCount` fix in `RewardEngine.sol` attempts to scale the E_scarcity ceiling by K (the number of compressed epochs) when multiple epochs are finalized atomically via `autoFinalizeEpochs()`. However, this scaling is fatally incompatible with a hard, immutable circuit-breaker embedded inside the `PSRE.sol` ERC-20 token contract itself.

PSRE.sol enforces a per-epoch mint rate limiter at Line 50:

```
None
uint256 public constant MAX_MINT_PER_EPOCH = 25_200e18; // 0.002 * 12,600,000
```

And in the mint() function at Line 133-138:

```
None
// Epoch rate limiter (circuit-breaker)
uint256 epochId = _currentEpochId();
require(
    epochMinted[epochId] + amount <= MAX_MINT_PER_EPOCH,
    "PSRE: epoch mint cap exceeded"
);
```

Crucially, PSRE._currentEpochId() uses block.timestamp, the current wall-clock epoch; not the historical epoch being finalized by RewardEngine. When autoFinalizeEpochs() catches up on K=3 missed epochs, all three _finalizeSingleEpoch() calls execute in the same transaction, in the same block, under the same block.timestamp. Every mint call hits the same epochMinted[currentEpochId] counter.

If the total minting across all K compressed epochs exceeds 25,200 PSRE (which it will under any meaningful protocol activity with multiple vaults), the PSRE.mint() call

hard-reverts with "PSRE: epoch mint cap exceeded", permanently bricking autoFinalizeEpochs().

Attack Scenario

1. The protocol operates normally. Multiple PartnerVaults generate legitimate demand over several weeks.
2. A period of inactivity occurs (no buy() calls for K epochs). Demand accumulates.
3. A user calls buy() on any PartnerVault, which triggers autoFinalizeEpochs().
4. The RewardEngine correctly computes E_scarcity * K and attempts to mint the scaled reward pool.
5. PSRE.mint() reverts because epochMinted[currentEpoch] + scaledAmount > MAX_MINT_PER_EPOCH.
6. The entire buy() transaction reverts. No user can buy from any PartnerVault.
7. Manual finalizeEpoch() calls are the only fallback, but these enforce the 1x cap (Vulnerability 1), permanently destroying the compressed rewards.

Proof of Concept

The following test empirically triggers the DoS. Both tests use identical parameters (300,000 PSRE growth, 3 skipped epochs). Only the finalization path differs.

None
```
// ── Vuln 1: Manual Bypass -- succeeds but burns rewards ──────────────
    function test_Vuln1_ManualFinalizeBypass() public {
        PartnerVault pvHonest = _createVault(partner, INITIAL_PSRE);
        _finalizeEpoch(0);

        uint256 growth = 300000e18; // 300k PSRE
        _buyPSRE(partner, pvHonest, growth);
        vm.warp(genesis + 4 * 7 days + 1); // Now in Epoch 4

        // Manual bypass: works, but enforces rigid 1x cap
        re.finalizeEpoch(1);

        uint256 epoch1psreMinted = re.cumulativeRewardMinted(address(pvHonest));
        console.log("epoch1psreMinted (Manual Bypass/Griefing)", epoch1psreMinted
/ 1e18);
        // Output: 8820 -- severely truncated
    }

    // ── Vuln 1b: Auto path -- REVERTS, protocol bricked ─────────────────
```

```
function test_Vuln1b_AutoFinalizeIntendedPath() public {
    PartnerVault pvHonest = _createVault(partner, INITIAL_PSRE);
    _finalizeEpoch(0);

    uint256 growth = 300000e18; // Identical 300k growth
    _buyPSRE(partner, pvHonest, growth);
    vm.warp(genesis + 4 * 7 days + 1); // Now in Epoch 4

    // Native auto path: REVERTS with "PSRE: epoch mint cap exceeded"
    re.autoFinalizeEpochs();
}
```

Foundry Output:

[PASS] test_Vuln1_ManualFinalizeBypass()
  Logs: epoch1psreMinted (Manual Bypass/Griefing) 8820

[FAIL: PSRE: epoch mint cap exceeded] test_Vuln1b_AutoFinalizeIntendedPath()

Impact

This vulnerability creates a complete protocol deadlock under realistic growth conditions:

Path	Outcome	Reward Loss
autoFinalizeEpochs()	Hard revert → protocol-wide DoS. No buy() possible.	100% — protocol frozen
finalizeEpoch() manual	Succeeds but enforces 1x cap (Vuln 1)	~66-93% of compressed rewards permanently burned

The developer's _autoFinalizeCount fix is therefore a lose-lose trap: the "fixed" path (autoFinalizeEpochs) reverts under real load, and the only working fallback (finalizeEpoch) silently destroys Partner rewards.

Root Cause:
The RewardEngine was modified to scale its internal ceiling (E_scarcity * K) without coordinating with the PSRE token's independent, immutable epoch mint cap. The PSRE.sol contract has no concept of _autoFinalizeCount or historical epoch finalization. It only knows block.timestamp, meaning all batch-finalized epochs share a single mint budget.

Recommendation

1. Remove _autoFinalizeCount scaling entirely, it is architecturally incompatible with the token's safety invariant.
2. Re-architect PartnerVault.sol to track historical time-weighted cumS per epoch, so that snapshotEpoch() returns only the growth attributable to a single epoch regardless of when it is called.
3. If batch finalization must be preserved, the PSRE.mint() call must be made epoch-aware: pass the historical epochId being finalized so that each epoch's mint is charged against its own budget, not the current wall-clock epoch.

Affected Code:

1. Non-epoch-aware delta in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:351-363
function snapshotEpoch() external override onlyRewardEngine returns (uint256
deltaCumS) {
    _updateCumS();
    if (cumS > lastEpochCumS) {
        deltaCumS = cumS - lastEpochCumS; // Captures ALL growth since last call
    } else {
        deltaCumS = 0;
    }
    lastEpochCumS = cumS; // Baseline is moved to current peak immediately
}
```

2. EMA update in `RewardEngine.sol`

```
None
// contracts/periphery/RewardEngine.sol:553-555
uint256 R_new = (R_old * (PRECISION - theta) + delta * theta) / PRECISION;
sumR = sumR - R_old + R_new;
R[vault] = R_new;
```

Proof of Concept:

- **Steady Growth**: A vault grows by **10 PSRE** every epoch for 5 epochs. If finalized on time, R climbs steadily toward a steady-state of ≈ 10.
- **Compressed Growth**: The same vault grows by **10 PSRE** every epoch, but `finalizeEpoch` is not called for 5 epochs.
- **The Spike**: `finalizeEpoch(0)` is finally called. `snapshotEpoch()` returns **50 PSRE** ($\Delta = 50$).
- **Calculation**:

$$R_{new} = (R_{old} \cdot 0.923) + (50 \cdot 0.077) \approx R_{old} + 3.85$$

- **Tier Promotion**: If the vault's R was **2** and the Silver threshold is **5**, this single spike pushes the vault into the Silver tier instantly.
- **Unearned Advantage**: For the next ≈ 13 epochs, the vault enjoys the Silver multiplier (**1.0x** instead of **0.8x**), despite its actual average growth being identical to a Bronze vault.

Impact:

This creates an economic imbalance where "bursty" growth or delayed finalization is rewarded more highly than consistent, timely growth. While rewards aren't exactly "stolen," they are redistributed from disciplined partners to those who benefit (intentionally or not) from compressed lapping. It undermines the intent of the EMA as a measure of consistent performance.

Recommendation:

The system should ensure that a single update cannot artificially inflate the EMA regardless of when the finalization occurs.

1. **Epoch-Aware Snapshots**: Modify `snapshotEpoch(uint256 epochId)` to take the epoch ID and return the growth specifically for that window. This requires `PartnerVault` to store a history of $cumS$ at each epoch boundary.
2. **Capped Deltas**: Implement a maximum allowable Δ per epoch in the `RewardEngine`. Any growth exceeding the cap is either discarded or queued for subsequent epochs.
3. **Normalized EMA**: If multi-epoch finalization is detected, the `RewardEngine` should distribute the total Δ across the missed epochs and update the EMA K times sequentially (one for each missed epoch) to simulate steady-state growth.

Status: Resolved

commit 177c18e

Issue #23: Stranded PSRE when an epoch is finalized with zero stakers on both sides

Severity: MEDIUM

Description:

`_finalizeSingleEpoch` mints `mintAmount` PSRE to RewardEngine, forceApproves StakingVault for `actualStakerMint = mintAmount - P_partners`, then calls `stakingVault.distributeStakerRewards`. When the snapshot captured zero stakers on both sides, StakingVault short-circuits and returns without pulling the tokens. Neither StakingVault's `sweepUnclaimedPool` (which requires `epochPSREPool[e] > 0`) nor RewardEngine (which has no sweep/rescue function) can recover the stranded amount and the mint has already been counted against the 12.6M `S_EMISSION` cap (`T += mintAmount`).

`contracts/periphery/RewardEngine.sol:694-711`

```
None
if (mintAmount > 0) {
    require(!IPausableToken(address(psre)).paused(), "RE: PSRE transfers
paused");
    psre.mintForEpoch(address(this), mintAmount, epochId);
    T += mintAmount;                        // ← cap consumed
    assert(T <= S_EMISSION);
}
...
uint256 actualStakerMint = mintAmount > P_partners ? mintAmount - P_partners :
0;
if (actualStakerMint > 0) {
    IERC20(address(psre)).forceApprove(address(stakingVault),
actualStakerMint);
    stakingVault.distributeStakerRewards(epochId, actualStakerMint);
}
```

`contracts/periphery/StakingVault.sol:274-293`

```
None

function distributeStakerRewards(uint256 epochId, uint256 totalStakerPool)
    external override onlyRewardEngine
{
    require(epochSnapshotted[epochId],  "StakingVault: not snapshotted");
    require(!epochDistributed[epochId], "StakingVault: already distributed");
    require(totalStakerPool > 0, "StakingVault: zero pool");

    uint256 totalPSREcheck = epochTotalPSREStaked[epochId];
    uint256 totalLPcheck   = epochTotalLPStaked[epochId];

    // No stakers at all: mark distributed, don't pull tokens.
    // Pool stays in caller (RewardEngine) — use sweepUnclaimedPool() if
needed.
    if (totalPSREcheck == 0 && totalLPcheck == 0) {
        epochDistributed[epochId] = true;
        ...
        return;                 // ← no safeTransferFrom
    }
```

`sweepUnclaimedPool` only reads StakingVault's own `epochPSREPool[e] / epochLPPool[e]`, which are 0 in this branch, so it cannot recover. RewardEngine contains no sweep / rescue / emergencyTransfer entry point for PSRE.

Impact

Permanent loss of emission headroom; the `actualStakerMint` tokens sit on RewardEngine's balance and consume the hard-coded 12.6M emission budget `S_EMISSION` without ever paying anyone. Reachable in practice during early protocol life (no stakers yet) or any period of staker inactivity aligning with a finalization both reachable via the permissionless `autoFinalizeEpochs.` No attacker-extractable value; the harm is protocol-level accounting drift.

Recommendation

Skip minting the staker share when the snapshot captured zero stakers, since there is no legitimate recipient and no recovery path.

```
None
// ── Staker pool ──────────────────────────
-    uint256 P_stakers = B_stakers;
+    uint256 P_stakers = B_stakers;
+    // No stakers at snapshot → nobody can claim; do not mint into an
unrecoverable bucket.
+    if (IStakingVault(stakingVault).epochTotalPSREStaked(epochId) == 0
+        && IStakingVault(stakingVault).epochTotalLPStaked(epochId) == 0) {
+        P_stakers = 0;
+    }

     uint256 P          = P_partners + P_stakers;
     uint256 mintAmount = P < remaining ? P : remaining;
```

Alternative: add an owner-only `rescuePSRE(uint256 amount, address to)` on RewardEngine scoped to strictly orphan balance (`balanceOf(this) - Σ owedPartner - unclaimedStakerApprovals`).

Status: Resolved

commit 4ca5bfc

Low Severity Issues

Issue #24: RewardEngine.pause() Does Not Gate claimPartnerReward and claimStake, Emergency Freeze Cannot Prevent Reward Outflows

Severity: LOW

Description:

The `RewardEngine` contract implements a pausing mechanism (via `PausableUpgradeable`) to allow the administrator to halt protocol operations during emergencies. However, the `whenNotPaused` modifier is only applied to the `finalizeEpoch` function.

The critical functions for withdrawing funds `claimPartnerReward` and `claimStake` do not implement the `whenNotPaused` check. This creates a vulnerability where an administrator can pause the contract to stop new epochs from being finalized, but cannot prevent the drainage of already-accrued rewards from the `owedPartner` and staker pools. In a scenario where a bug is discovered that has incorrectly inflated reward balances, the administrator cannot freeze the outflows to prevent an exploit while a fix is being deployed.

Affected Code:

1. `finalizeEpoch` correctly gated

```
None
// contracts/periphery/RewardEngine.sol:442
function finalizeEpoch(
    uint256 epochId
) external nonReentrant whenNotPaused {
     // ...
}
```

2. `claimPartnerReward` missing gate

```
None
// contracts/periphery/RewardEngine.sol:723
```

```
function claimPartnerReward(address vault) external nonReentrant {
    // missing whenNotPaused modifier
    require(factory.partnerOf(vault) == msg.sender, "RE: not vault owner");
    // ...
}
```

3. `claimStake` missing gate

```
None
// contracts/periphery/RewardEngine.sol:741
function claimStake(uint256 epochId) external nonReentrant {
    // missing whenNotPaused modifier
    require(epochFinalized[epochId], "RE: epoch not finalized");
    // ...
}
```

Impact:

While the pause mechanism successfully stops the creation of new rewards (by blocking `finalizeEpoch`), it fails to protect the existing reward pool. This limits the administrator's ability to respond to critical vulnerabilities that affect existing balances, potentially leading to loss of funds during the window between discovery and remediation.

Recommendation:

Add the `whenNotPaused` modifier to all functions that allow the extraction of funds from the `RewardEngine`.

Proposed Change in **`RewardEngine.sol`**:

```
None
function claimPartnerReward(address vault) external nonReentrant whenNotPaused {
    // ...
}

function claimStake(uint256 epochId) external nonReentrant whenNotPaused {
    // ...
```

```
}
```

Status: **Resolved**

commit 303cd2f

Issue #25: In-Loop `sumR` Mutation Causes Order-Dependent Tier Assignments

Severity: **LOW**

Description:
In RewardEngine.finalizeEpoch(), the vault processing loop computes each vault's EMA rolling score (R_{new}), immediately updates the global sumR accumulator, and then evaluates the vault's tier share ($s = R_{new} / sumR$) within the **same iteration** .

This creates a **"Sliding Denominator"** vulnerability. When the i-th vault is evaluated, the sumR used in its share calculation includes the updated scores for all preceding vaults ($0 \dots i$) but excludes the updated scores for all subsequent vaults ($i+1 \dots N-1$) .

Because factory.getAllVaults() returns vaults in their permanent registration order, earlier-registered partners receive a structural and immutable advantage. Their shares are calculated against a "partial" global sum, leading to inflated values and unearned tier promotions .

Vulnerability logic:

```
None
for (uint256 i = 0; i < nVaults; i++) {
    // ...
    uint256 R_old = R[vault];
    uint256 R_new = (R_old * (PRECISION - theta) + delta * theta) / PRECISION;

    // @audit: global sumR is updated BEFORE the share calculation
    sumR = sumR - R_old + R_new;
    R[vault] = R_new;
```

```
    if (sumR > 0) {
        // @audit: 's' is calculated using a partially updated sumR
        uint256 s = (R_new * PRECISION) / sumR;
        // ... tier assignment logic follows ...
    }
}
```

Proof of Concept:

Consider 100 vaults with identical performance. Each should technically have a **1% share** of the total rewards. Due to the sliding denominator, the calculated share for Vault i is:

s_i = frac{R□ew,i}{Σ□=0 i R□ew,□ + Σ□=i+1 N−1 Ro□ə,□}

Calculated Shares in a Uniform System:

- **Vault 0 (First):** s = R□ew / R□ew = 100% (Always Gold multiplier).
- **Vault 49 (Mid):** s ≈ R□ew / (50 · R□ew) = 2% (Threshold for Gold).
- **Vault 99 (Last):** s ≈ R□ew / (100 · R□ew) = 1% (Correct share, Silver tier).

Result: The first 50 identical vaults receive the **1.2x Gold multiplier**, while the last 50 receive the **1.0x Silver multiplier** solely due to their position in the array

Impact:

This bug creates a structural reward multiplier skew for identical economic activity:

Scenario (100 equal vaults)	Vault Position	During-loop s	Tier Assigned	Multiplier
Early Entry	i=0i=0	100%100%	**Gold**	**1.2x**
Mid Entry	i=49i=49	2.0%2.0%	**Gold**	**1.2x**
Mid Entry	i=50i=50	1.96%1.96%	**Silver**	**1.0x**
Late Entry	i=99i=99	1.0%1.0%	**Silver**	**1.0x**

The “Early Bird” Premium: The first 50 vaults receive a 20% reward premium over the last 50 vaults. This advantage is deterministic and immutable; because `allVaults[]` is never reordered, registration timing alone guarantees a permanent economic advantage without requiring additional capital or superior performance.

Recommendation:

Decouple the state update from the tier assignment by implementing a **Two-Pass Approach**. This ensures every vault's share is calculated against a fully settled and stable global `sumR`.

Proposed Refactor for RewardEngine.sol:

1. **Pass 1:** Iterate through all vaults to update individual R scores and compute the final, authoritative `sumR` for the epoch. Store the new R values in a temporary memory array.
2. **Pass 2:** Iterate through the memory array, calculating tiers and multipliers using the fixed `sumR` calculated in Pass 1.

```
None
// Pass 1: Compute final sumR
uint256 finalSumR = sumR;
for (uint256 i = 0; i < nVaults; i++) {
    uint256 R_new = computeEMA(vaults[i]);
    finalSumR = finalSumR - R[vaults[i]] + R_new;
    R[vaults[i]] = R_new;
    tempR[i] = R_new;
}
sumR = finalSumR; // Commit settled sumR

// Pass 2: Assign tiers using stable denominator
for (uint256 i = 0; i < nVaults; i++) {
    uint256 s = (tempR[i] * PRECISION) / sumR;
    // ... assign multipliers ...
}
```

Status: **Resolved**

commit d2d04ad

Issue #26: Multi-Epoch Accumulated StakeTime Permanently Destroyed by `recordStakeTime`'s Blanket `accStakeTime` Reset

Severity: Low

Description:

When a user calls `recordStakeTime(epochId)`, the function unconditionally resets the entire `accStakeTime` accumulator to zero after recording only a single epoch's worth of time. Because the accumulator is a single shared counter spanning all past epochs since the user's last interaction, this blanket reset permanently destroys the historical accumulated `stakeTime` for every other past epoch the user never recorded.

From `StakingVault.sol`:

```
None

// Step 1: _checkpointUser fires and dumps ALL elapsed time (e.g., 10 epochs) into
accStakeTime
s.accStakeTime += totalStake * elapsed; // accumulator = 10 * epochDuration worth
of weight

// Step 2: Cap the recorded amount to a single epoch
uint256 st = s.accStakeTime < maxEpochStakeTime
    ? s.accStakeTime
    : maxEpochStakeTime;  // st = 1 epoch's worth

userStakeTimeByEpoch[epochId][msg.sender] = st; // Only epoch 0 recorded

// Step 3: THE BLANKET DESTRUCTION
// @audit: should be `s.accStakeTime -= st` to preserve the remaining 9 epochs
s.accStakeTime = 0; // 9 epochs of legitimate staking weight vapourised
```

Because `lastUpdateTimestamp` is also updated to `block.timestamp` inside `_checkpointUser`, any subsequent calls to `recordStakeTime` for other epochs in the same transaction or shortly after will calculate `elapsed = ~0 seconds`, producing `delta = 0`. The user's remaining 9 past epochs are permanently unclaimable.

Proof of Concept:

1. **Staking:** User A stakes 100 PSRE at Epoch 0. They interact with the protocol for no reason for 10 weeks, silently staking across Epochs 0-9.
2. **Recording Epoch 0:** At Epoch 10, User A calls `recordStakeTime(0)`. `_checkpointUser` runs, correctly computing 10 weeks of accumulated time and storing it all into `accStakeTime`. The cap mechanism saves 1 epoch's worth into `userStakeTimeByEpoch[0]`, and then `s.accStakeTime = 0` permanently destroys the remaining 9 weeks.
3. **Recording Epoch 1 (Same Block):** User A immediately calls `recordStakeTime(1)`. Inside `_checkpointUser, elapsed = block.timestamp - s.lastUpdateTimestamp = 0 seconds. delta = 0`. The accumulator remains at 0. `userStakeTimeByEpoch[1][user] = 0`.
4. **Impact:** User A legitimately staked for 10 epochs but can only ever claim a reward for 1 of them. 9 epochs of staking rewards are permanently destroyed and unclaimable.

Impact:

Direct loss of funds for any user who does not call `recordStakeTime` after every single epoch. A single missed week makes all prior unclaimed epochs permanently unclaimable. This also creates a perverse incentive where the optimal user behaviour is to call `recordStakeTime` on the exact epoch boundary every single 7-day period with no tolerance for latency.

Note:
The issue is partially mitigated by the introduction of autoFinalization(). During the time of inactivity via buy() or createVault() , the issue will persist.

Recommendation:

Change the blanket reset to a targeted subtraction so that only the recorded epoch's allocation is consumed from the accumulator:

```
None

- s.accStakeTime = 0;
+ s.accStakeTime -= st;
```

This change ensures the remaining accumulated `stakeTime` for past unrecorded epochs is preserved inside the accumulator and can be correctly claimed when the user calls `recordStakeTime` for those respective epochs.

Optionally , enforce sequential recording order:

```
None
require(
    epochId == 0 || userStakeTimeByEpoch[epochId - 1][msg.sender] > 0,
    "StakingVault: record previous epoch first"
);
```

This prevents out-of-order calls from silently destroying earlier epochs.

Status: Resolved

commit 303cd2f

Issue #27: `PartnerVault.qualified` Flag Permanently False Due to Lack of Synchronization with `RewardEngine`

Severity: LOW

Description:

The `PartnerVault` contract contains a `qualified` state variable, which is explicitly initialized to `false` during `factoryInit()`. This variable is exposed via the `isQualified()` public view function, as defined in the `IPartnerVault` interface.

However, the logic that determines whether a vault has actually qualified for rewards is implemented within the `RewardEngine` contract. In `RewardEngine.finalizeEpoch()`, the engine checks if a vault's `cumS` has grown past its `initialCumS` and, if so, sets a separate `qualified[vault]` mapping to `true` within the **RewardEngine's own storage**.

There is no mechanism in the system for the `RewardEngine` to call back into the `PartnerVault` to update its `qualified` flag. Consequently, while the `RewardEngine` correctly tracks qualification for reward distribution, the `PartnerVault` state remains permanently `false`.

Affected Code:

1. Initialization in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:175
function factoryInit(uint256 psreOut) external onlyFactory {
    // ...
    qualified = false;
}
```

2. Qualification logic in `RewardEngine.sol`

```
None
// contracts/periphery/RewardEngine.sol:527
if (currentCumS > initialCumS[vault]) {
    // Vault qualifies for the first time
    qualified[vault] = true; // This updates RewardEngine storage, NOT
PartnerVault storage
    // ...
}
```

3. Public interface in `PartnerVault.sol`

```
None
// contracts/core/PartnerVault.sol:425
function isQualified() external view returns (bool) {
    return qualified;
}
```

Proof of Concept:

1. **Setup**: A partner creates a `PartnerVault`. `PV.qualified` is set to `false`.
2. **Growth**: The partner increases the ecosystem balance such that `cumS > initialCumS`.

3. **Finalization**: The `RewardEngine` calls `finalizeEpoch()`. It detects the growth and sets `RE.qualified[vault] = true`.
4. **Verification**:
    - An external entity calls `RewardEngine.qualified(vault)`, which returns **true**.
    - The same entity calls `PartnerVault.isQualified()`, which returns **false**.
5. **Result**: The public API of the `PartnerVault` provides permanently incorrect data regarding the vault's qualification status.

Recommendation:

To ensure consistency, the `PartnerVault` should not maintain its own `qualified` state. Instead, the `isQualified()` function should be updated to query the `RewardEngine` directly.

Proposed Change in **`PartnerVault.sol`**:

```
None
function isQualified() external view returns (bool) {
     return IRewardEngine(rewardEngine).qualified(address(this));
}
```

This removes the redundant storage variable and ensures that the `PartnerVault` always reports the authoritative state maintained by the `RewardEngine`.

Status: Resolved

commit 44f17d8

Post Fixes Issues

Issue #28: Stale `partnerOwner` in `CustomerVault` after vault ownership transfer

Severity: LOW

Description:

`CustomerVault.sol : 115,131-144`

`partnerOwner` set once at `init`, never updated. After PV ownership transfer, old owner retains `claimVault()` authority on all existing CVs. Can redirect platform-managed CV funds to the attacker wallet.

Recommendation:

Fix: either check `IPartnerVault(parentVault).owner()` in `claimVault`

Status: Resolved

commit 31eb313

Informational Severity Issues

Issue #29: Governance Parameter Queue Functions Allow Silent Value Overwrite Without Cancellation Event

Severity: INFORMATIONAL

Description:

The `RewardEngine` contract implements a timelock for governance parameter updates. Parameters such as `alphaBase`, `E0`, and `partnerSplit` are first queued via `queue` functions and can only be applied after `PARAM_TIMELOCK` (48 hours) has elapsed.

However, there is a significant inconsistency in how these queues are managed compared to the UUPS upgrade queue. The `scheduleUpgrade` function explicitly prevents overwriting a pending upgrade by requiring `pendingUpgrade == address(0)`. In contrast, the parameter queue functions (`queueAlphaBase`, `queueE0`, `queuePartnerSplit`, and `queueTierParams`) unconditionally overwrite any existing `PendingParam` and reset the timelock without requiring a prior cancellation.

This enables a “bait-and-switch” governance tactic where a favorable value is queued to gain community approval, only to be silently replaced by a less favorable value shortly before the original timelock expires.

Affected Code:

1. Unconditional overwrite in parameter queues

```
None
// contracts/periphery/RewardEngine.sol:766-769
function queueAlphaBase(uint256 v) external onlyOwner {
    require(v >= ALPHA_MIN && v <= ALPHA_MAX, "RE: out of bounds");
    // Overwrites existing pendingAlphaBase without check
    pendingAlphaBase = PendingParam(v, block.timestamp + PARAM_TIMELOCK);
    emit ParamUpdateQueued("alphaBase", v, pendingAlphaBase.readyAt);
}
```

2. Guarded state in upgrade queue

```
None
// contracts/periphery/RewardEngine.sol:321-323
require(
    pendingUpgrade == address(0),
    "RE: upgrade already scheduled - cancel first"
);
pendingUpgrade = newImplementation;
```

Proof of Concept:

1. **Initial Queue**: The owner calls `queueAlphaBase(0.05e18)` (minimum rate). A `ParamUpdateQueued` event is emitted.
2. **Community Approval**: Stakeholders observe the event and approve of the low rate.
3. **Silent Overwrite**: At hour 47, the owner calls `queueAlphaBase(0.15e18)` (maximum rate).
4. **State Change**: `pendingAlphaBase` is updated to **0.15e18** and the `readyAt` timer is reset to 48 hours from the current block.
5. **Outcome**: Observers who only tracked the first event see the value changed without an explicit "Cancel" event, and the transition period for the new value is restarted, potentially confusing the governance process.

Impact:

While a new event is emitted, the lack of a `require` guard allows the owner to pivot parameters without explicitly acknowledging the abandonment of the previous commitment. This weakens the transparency of the governance process compared to the strict requirements applied to contract upgrades.

Recommendation:

Align the parameter queue logic with the upgrade queue logic:

- **Add a Pending Check**: Require `pendingValue.readyAt == 0` before allowing a new value to be queued.
- **Implement Cancellation**: Provide a `cancelParam` function to allow the owner to explicitly clear a pending update and emit a `ParamUpdateCancelled` event.

Status: Resolved

commit 7c13690

Disclaimer

The smart contracts provided by the client with the purpose of security review have been thoroughly analyzed in compliance with the industrial best practices till date w.r.t. Smart Contract Weakness Classification (SWC) and Cybersecurity Vulnerabilities in smart contract code, the details of which are enclosed in this report.

This report is not an endorsement or indictment of the project or team, and they do not in any way guarantee the security of the particular object in context. This report is not considered, and should not be interpreted as an influence, on the potential economics of the token (if any), its sale, or any other aspect of the project that contributes to the protocol's public marketing.

Crypto assets/ tokens are the results of the emerging blockchain technology in the domain of decentralized finance and they carry with them high levels of technical risk and uncertainty. No report provides any warranty or representation to any third-party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the reports in any way, including to make any decisions to buy or sell any token, product, service, or asset. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and is not a guarantee as to the absolute security of the project.

Smart contracts are deployed and executed on a blockchain. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. The scope of our review is limited to a review of the programmable code and only the programmable code, we note, as being within the scope of our review within this report. The smart contract programming language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer or any other areas beyond the programming language's compiler scope that could present security risks.

This security review cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While BlockApex has done their best in conducting the analysis and producing this report, it is important to note that one should not rely on this report only - we recommend proceeding with several independent code security reviews and a public bug bounty program to ensure the security of smart contracts.